

TRABAJO DE FIN DE MÁSTER

Máster en Ciberseguridad

Implementación de un Sistema de Control de Acceso

Basado en el Modelo Zero Trust

Paula Escudero García

Curso académico 2025-2026



Unión Europea

SEDE PORVENIR - VIAPOL: C/ Isabela, 1 - 41013 Sevilla SEDE CARTUJA: C/ Leonardo Da Vinci, 7 - 41092 Sevilla

ÍNDICE

Calendario del Proyecto	4
Calendario Estimado	4
Calendario Real	5
1. Introducción	5
2. Marco Teórico	6
2.1. El Modelo de Seguridad Tradicional y sus Limitaciones	6
2.2. Origen y Evolución del Modelo Zero Trust	7
2.3. Principios Fundamentales del Zero Trust	8
2.4. Los Cinco Pilares del Zero Trust	9
2.4.1. Identidad	9
2.4.2. Dispositivo	10
2.4.3. Red	10
2.4.4. Aplicación y Carga de Trabajo	10
2.4.5. Datos	10
2.5. Zero Trust Network Access (ZTNA)	11
2.6. Casos de Uso y Adopción en la Industria	11
2.7. Herramientas y Tecnologías para Implementar Zero Trust	12
2.7.1. Control de Acceso Remoto: OpenSSH	12
2.7.2. Prevención de Intrusiones: Fail2ban	13
2.7.3. Centralización de Logs: rsyslog y Wazuh	13
2.7.4. Control de Cortafuegos: UFW	13
2.7.5. Autenticación Multifactor: Google Authenticator (TOTP)	14
2.8. Comparativa de Soluciones	14
2.9. Marco Normativo y Estándares Relacionados	15
2.9.1. NIST SP 800-207 — Zero Trust Architecture	15
2.9.2. Esquema Nacional de Seguridad (ENS)	16
2.9.3. ISO/IEC 27001 — Gestión de la Seguridad de la Información	16
2.9.4. CIS Controls	16
3. Diseño de la Infraestructura Virtual	16
3.1. Visión general del entorno	16
3.2. Topología de red y diagrama	17
3.3. Roles de cada nodo	18
3.3.1. Servidor de recursos (Target) — 192.168.10.20	18
3.3.2. Central de logs (Nodo de monitorización) — 192.168.10.10	18
3.3.3. Nodo de auditoría (Cliente / Atacante) — 192.168.10.30	19



3.4. Especificaciones técnicas	19
3.5. Configuración de red en VMware	20
3.6. Inventario de software	20
3.7. Flujos de tráfico	21
3.7.1. Acceso legítimo	21
3.7.2. Ataque y respuesta automática	21
3.7.3. Telemetría de seguridad	22
3.8. Decisiones de diseño	22
3.9. Limitaciones del diseño	22
4. Implementación del Laboratorio	23
4.1. Despliegue de la Infraestructura Virtual	23
4.2. Configuración de Red y Segmentación	24
4.3. Fortalecimiento (Hardening) del Servicio OpenSSH	24
4.4. Configuración de la Monitorización Centralizada con rsyslog	25
4.4.1. Configuración del servidor rsyslog en la central	26
4.4.2. Configuración del agente rsyslog en el servidor	26
4.5. Implementación de Reglas de Respuesta Automática con Fail2ban	27
4.6. Instalación y Configuración de Wazuh	27
4.6.1. Instalación del stack completo	28
4.6.2. Acceso al panel web	28
4.7. Configuración del Cortafuegos con UFW	29
4.8. Verificación del Sistema Completo	30
4.9. Implementación de la Autenticación Multifactor (MFA) con TOTP	31
4.9.1. Instalación y configuración en el servidor	31
4.9.2. Configuración de PAM y sshd_config	31
4.9.3. Verificación del funcionamiento	32
5. Pruebas y Resultados	32
5.1. Escenario 1: Acceso Legítimo y Trazabilidad	33
5.1.1. Descripción del escenario	33
5.1.2. Ejecución	33
5.1.3. Verificación en la central	33
5.2. Escenario 2: Reconocimiento de Red con nmap	34
5.2.1. Descripción del escenario	34
5.2.2. Ejecución	34
5.2.3. Análisis de resultados	34
5.3. Escenario 3: Simulación de Ataque de Fuerza Bruta	35
5.3.1. Descripción del escenario	35
5.3.2. Ejecución del ataque	35



5.3.3. Respuesta automática de Fail2ban	35
5.3.4. Verificación del bloqueo	36
5.4. Escenario 4: Análisis de la Respuesta del Sistema	36
5.4.1. Registro de eventos en auth.log	36
5.4.2. Centralización en la central de logs	36
5.5. Escenario 5: Verificación de la Integridad de los Logs Centralizados	37
5.5.1. Descripción	37
5.5.2. Verificación	37
5.6. Escenario 6: Detección en el Panel Wazuh	37
5.6.1. Alertas generadas	38
5.6.2. Threat Hunting	38
5.6.3. Agente activo	39
5.7. Análisis Global de Resultados	39
5.7.1. Verificación continua de identidad	39
5.7.2. Mínimo privilegio	39
5.7.3. Respuesta automática ante anomalías	39
5.7.4. Trazabilidad completa	40
5.8. Plan de Respuesta ante Incidentes de Seguridad	40
5.8.1. Incidente 1: Compromiso de Credenciales SSH	40
5.8.2. Incidente 2: Ataque de Fuerza Bruta Distribuido	41
6. Análisis de Costes y Sostenibilidad	42
6.1. Coste del Entorno de Laboratorio	42
6.2. Coste de una Implementación Real en Producción	43
Escenario 1: Infraestructura on-premise	43
Escenario 2: Infraestructura en nube pública (AWS)	44
6.3. Comparativa con Soluciones Comerciales Equivalentes	45
6.4. Sostenibilidad	45
7. Conclusiones y Líneas Futuras	46
7.1. Conclusiones	46
7.1.1. Objetivos alcanzados	47
7.1.2. Lecciones aprendidas	47
7.1.3. Limitaciones del proyecto	48
7.2. Logros Adicionales y Líneas Futuras	48
7.2.1. Cifrado del canal de telemetría	48
7.2.2. Autenticación Multifactor (MFA) — Implementada	49
7.2.3. Evaluación de postura de dispositivo	49
7.2.4. Microsegmentación real	49
7.2.5. Integración con un directorio de identidades	49



7.2.6. Alertas automáticas y respuesta orquestada	50
7.3. Reflexión Final	50
8. Bibliografía y Anexos	50
8.1. Bibliografía	50
8.1.1. Estándares y marcos normativos	50
8.1.2. Publicaciones académicas y técnicas	51
8.1.3. Documentación técnica de herramientas	51
8.1.4. Recursos web complementarios	51
8.2. Anexos	52
Anexo A. Ficheros de configuración completos	52
Anexo B. Comandos de verificación del sistema	54
Anexo C. Glosario de términos	55
Anexo D. Capturas de pantalla	57

Calendario del Proyecto

Calendario Estimado

La planificación inicial del proyecto contempla cinco fases distribuidas entre enero y mayo de 2026. La primera fase, de recopilación de información y revisión bibliográfica, se estimó para enero. La segunda fase, de diseño del laboratorio y estudio del marco teórico, se planificó para febrero. La fase de implementación práctica se asignó a marzo y principios de abril. La documentación se planificaba para abril, dejando mayo para la revisión final y la entrega del trabajo.

Estimado	Ene 2026	Feb 2026	Mar 2026	Abr 2026	May 2026
Recopilación info					
Marco teórico					
Implementación					



Pruebas					
Documentación					
Revisión / Entrega					

Calendario Real

El desarrollo real del proyecto sufrió un retraso respecto a la planificación inicial. Durante enero y febrero la carga de trabajo en otros ámbitos impide dedicar el tiempo necesario al TFM, por lo que el arranque efectivo del proyecto no se produjo hasta marzo. A partir de ese momento el ritmo de trabajo fue intenso para compensar el tiempo perdido: la recopilación de información, el marco teórico, la implementación del laboratorio y las pruebas se solaparon en parte durante los meses de marzo y abril. La documentación se desarrolló en paralelo con las últimas fases prácticas, y la revisión y entrega final se concentraron en mayo.

Real	Ene 2026	Feb 2026	Mar 2026	Abr 2026	May 2026
Recopilación info					
Marco teórico					
Implementación					
Pruebas					
Documentación					
Revisión / Entrega					

Leyenda:

	Recopilación de información		Realización parte práctica
	Documentación		Revisión y entrega
	Alta carga / no pudo comenzar		



Unión Europea

SEDE PORVENIR - VIAPOL: C/ Isabela, 1 - 41013 Sevilla SEDE CARTUJA: C/ Leonardo Da Vinci, 7 - 41092 Sevilla

1. Introducción

El presente Trabajo de Fin de Máster aborda el diseño e implementación de un sistema de control de acceso corporativo fundamentado en el paradigma de seguridad Zero Trust. En un contexto en el que las organizaciones afrontan amenazas cada vez más sofisticadas, el perímetro de red tradicional ha dejado de ser una barrera de protección suficiente. La proliferación del trabajo remoto, la adopción masiva de entornos cloud y la creciente complejidad de las infraestructuras híbridas exigen un nuevo enfoque: verificar continuamente, no confiar nunca por defecto. Incidentes como el ataque a SolarWinds en 2020, que comprometió a cientos de organizaciones aprovechando la confianza implícita de las redes internas, o las brechas masivas derivadas de credenciales comprometidas, ponen de manifiesto que los modelos de seguridad basados en perímetro ya no son suficientes en el entorno actual.

Este proyecto propone y despliega, sobre un entorno de servidores virtuales Linux, una arquitectura en la que todo acceso queda validado en el momento en que se produce, todo evento queda registrado de forma centralizada y el sistema reacciona de manera automática ante comportamientos anómalos. El resultado es una plataforma funcional, trazable y adaptable que sirve como demostración práctica de cómo puede aplicarse la seguridad moderna en una organización real, utilizando exclusivamente herramientas de código abierto y sin necesidad de inversión económica en licencias propietarias. El objetivo principal del proyecto es implementar y validar los principios fundamentales del modelo Zero Trust en un entorno controlado, demostrando que conceptos como la autenticación multifactor, el mínimo privilegio, la microsegmentación lógica de red y la monitorización continua no son exclusivos de grandes corporaciones con presupuestos elevados, sino que son alcanzables con herramientas accesibles y conocimiento técnico adecuado. Los objetivos específicos incluyen: configurar un servicio SSH con hardening estricto y autenticación de dos factores mediante TOTP; implementar respuesta automática ante ataques mediante Fail2ban; centralizar la telemetría de seguridad en un nodo independiente mediante rsyslog; desplegar la plataforma SIEM Wazuh para correlación de eventos y detección de amenazas; y validar el conjunto mediante escenarios de prueba que cubren tanto accesos legítimos como ataques simulados. El documento se estructura en ocho capítulos. El capítulo 2 establece el marco teórico sobre el modelo Zero Trust y las herramientas utilizadas. El capítulo 3 describe el diseño de la infraestructura virtual. El capítulo 4 detalla la implementación paso a paso de todos los componentes. El capítulo 5 documenta las pruebas y resultados obtenidos. El capítulo 6 analiza los costes y la sostenibilidad de la solución. El capítulo 7 presenta las conclusiones y líneas futuras. El capítulo 8 recoge la bibliografía y los anexos técnicos.



2. Marco Teórico

2.1. El Modelo de Seguridad Tradicional y sus Limitaciones

Durante décadas, cuando pensábamos en seguridad informática en una empresa, la imagen mental era siempre la misma: un perímetro. Un castillo rodeado de foso, como se le llama habitualmente en el sector (*castle-and-moat*). La idea era sencilla: todo lo que está dentro de la red corporativa es de confianza, y lo que viene de fuera, no. Los esfuerzos se concentraban en vigilar bien ese borde: cortafuegos, sistemas de detección de intrusiones, VPNs.

Y durante mucho tiempo funcionó. Cuando los empleados trabajaban desde equipos fijos en la oficina y los servidores estaban en el sótano del edificio, ese modelo tenía sentido. El problema es que el mundo cambió, y el modelo no lo hizo al mismo ritmo. Cuatro tendencias lo fueron erosionando hasta dejarlo obsoleto:

El teletrabajo y la movilidad rompieron la idea de que "dentro de la red" significa "lugar seguro". Hoy un empleado puede conectarse desde casa, desde una cafetería o desde el aeropuerto, usando redes que la empresa no controla en absoluto.

La adopción masiva del cloud hizo que los datos y las aplicaciones dejarán de estar en un servidor físico al que se puede poner un cortafuegos. Están distribuidos entre AWS, Azure, Google Cloud, aplicaciones SaaS... ¿Dónde está el perímetro ahora?

Las amenazas internas demostraron que el modelo asume algo que no es verdad: que todo usuario autenticado dentro de la red es de confianza. La realidad es que una parte significativa de los incidentes de seguridad viene de dentro, ya sea por empleados descontentos o, más frecuentemente, por cuentas comprometidas.

Y quizás el punto más crítico: el movimiento lateral. Cuando un atacante consigue entrar por cualquier resquicio, el modelo perimétrico no le pone ningún obstáculo para moverse libremente por toda la red interna. Si tienes acceso, tienes acceso a todo.

El resultado es que la confianza implícita que el modelo tradicional otorga a todo lo que está "dentro" dejó de ser una ventaja para convertirse en una debilidad estructural. Era necesario replantearlo desde cero.



2.2. Origen y Evolución del Modelo Zero Trust

La respuesta a ese problema tiene nombre y apellido: John Kindervag, analista de Forrester Research, que en 2010 acuñó el término *Zero Trust* y formuló su premisa central de una manera que no deja lugar a interpretaciones: *nunca confíes, siempre verifica*. En lugar de asumir que alguien merece acceso por el simple hecho de estar en la red correcta, el modelo exige verificar cada solicitud de acceso, siempre, independientemente de de dónde venga.

Lo interesante es que la idea no surgió de la nada. Sus raíces intelectuales son más antiguas. El principio de *mínimo privilegio*, formulado por Saltzer y Schroeder ya en 1975, establece algo tan sentido común como esto: cada parte del sistema solo debe tener los permisos estrictamente necesarios para hacer su trabajo, nada más. Es uno de los pilares sobre los que se construye Zero Trust.

En 2013, Google hizo algo que marcó un antes y un después: publicó los detalles de *BeyondCorp*, su propio modelo de seguridad, que llevaba años funcionando internamente. La idea era radical para su época: el nivel de confianza no debería depender de dónde estés en la red, sino de quién eres y desde qué dispositivo te conectas. BeyondCorp fue la primera demostración pública a gran escala de que Zero Trust no era solo teoría, sino algo que se podía implementar de verdad.

La consolidación definitiva llegó en 2020, cuando el NIST (el organismo de estándares tecnológicos de Estados Unidos) publicó la guía SP 800-207, *Zero Trust Architecture*. Este documento es hoy la referencia técnica por excelencia: define los componentes de una arquitectura Zero Trust, los principios que deben guiar su diseño y los diferentes modelos de implementación. Es el marco que este proyecto toma como referencia principal.

Para cerrar el círculo, en 2021 la administración Biden firmó una orden ejecutiva exigiendo a todas las agencias federales americanas adoptar una arquitectura Zero Trust. Cuando los gobiernos empiezan a legislarlo, es que ya no es una tendencia, es el estándar.

2.3. Principios Fundamentales del Zero Trust

Una cosa que hay que tener clara desde el principio es que Zero Trust no es un producto que se compra ni una herramienta que se instala. Es una forma de pensar la seguridad, una filosofía de diseño que se articula en torno a siete principios que el NIST SP 800-207



recoge de forma formal. Merece la pena entenderlos bien porque son los que guían cada decisión de implementación en este proyecto:

- Todas las fuentes de datos y servicios de computación se consideran recursos: El modelo Zero Trust amplía el concepto de recurso corporativo más allá de los dispositivos físicos de la empresa, incluyendo dispositivos personales de empleados, servicios cloud y dispositivos IoT.
- Todas las comunicaciones deben ser seguras independientemente de la ubicación de la red: La seguridad no puede depender de que el tráfico discurra por la red interna. Toda comunicación debe estar cifrada y autenticada.
- El acceso a los recursos se concede en función de la sesión: El acceso no se otorga de manera permanente sino de forma puntual para cada sesión concreta. Una vez que la sesión finaliza, el acceso se revoca.
- El acceso a los recursos está determinado por políticas dinámicas: Las decisiones de acceso tienen en cuenta múltiples atributos: identidad del usuario, estado del dispositivo, ubicación, comportamiento histórico y el nivel de sensibilidad del recurso solicitado.
- La organización monitoriza y mide la integridad y la postura de seguridad de todos los activos propios y asociados: Los activos deben ser evaluados continuamente para detectar vulnerabilidades, configuraciones incorrectas o comportamientos anómalos.
- Toda autenticación y autorización es dinámica y se aplica estrictamente antes de que se permita el acceso: El sistema debe poder revocar el acceso en tiempo real si detecta un cambio en las condiciones de seguridad.
- La organización recopila tanta información como sea posible sobre el estado de los activos, la infraestructura de red y las comunicaciones, y la utiliza para mejorar su postura de seguridad: La recopilación y correlación de logs y telemetría es un componente esencial del modelo.

2.4. Los Cinco Pilares del Zero Trust

Para que todo esto no se quede en teoría, la industria ha organizado los principios anteriores en cinco pilares que cubren los vectores de ataque más importantes. Son los dominios sobre los que hay que trabajar para construir una arquitectura Zero Trust real.

2.4.1. Identidad

La identidad es el pilar central del modelo Zero Trust. Si ya no hay un perímetro de red que separe el "dentro" del "fuera", la identidad del usuario pasa a ser el nuevo perímetro. La pregunta ya no es "¿está esta persona en nuestra red?" si no "¿sabemos con certeza quién es esta persona?".



Esto significa que confiar solo en una contraseña no es suficiente. Las contraseñas se roban, se filtran, se adivinan. El modelo Zero Trust exige autenticación multifactor (MFA): algo que sabes (contraseña), más algo que tienes (un código temporal en tu móvil, una clave física). Así, aunque alguien consiga tu contraseña, no puede entrar sin ese segundo factor. En este proyecto se implementa MFA mediante Google Authenticator, que genera códigos TOTP según el estándar RFC 6238.

2.4.2. Dispositivo

La identidad del usuario no es suficiente por sí sola. Un usuario legítimo conectado desde un portátil infectado con malware es tan peligroso como un atacante externo. Por eso el segundo pilar pregunta: ¿desde qué dispositivo se está haciendo esta solicitud? ¿Está actualizado? ¿Cumple las políticas de la organización? ¿Tiene algún indicador de compromiso?

En entornos empresariales maduros esta evaluación se automatiza con herramientas MDM o EDR. En este laboratorio, la verificación del dispositivo se implementa a través de las claves SSH autorizadas: sólo los dispositivos que tienen la clave correcta configurada pueden intentar siquiera autenticarse.

2.4.3. Red

En Zero Trust, la red interna no es de confianza. Es simplemente el cable que transporta los datos, y hay que tratarlo con el mismo escepticismo que la red pública. La técnica principal aquí es la microsegmentación: dividir la red en segmentos pequeños y aislados para que, si un atacante compromete uno, no pueda moverse al resto. En este proyecto se implementa mediante UFW con política de denegación por defecto: todo el tráfico está bloqueado salvo el que se permite explícitamente.

2.4.4. Aplicación y Carga de Trabajo

Las aplicaciones tampoco son accesibles directamente desde la red. En una arquitectura Zero Trust el acceso pasa siempre por un punto de control que verifica identidad y privilegios antes de dejar pasar la solicitud. Un usuario solo accede a lo que necesita para su trabajo, y solo mientras lo necesita. Los accesos administrativos, que son los más sensibles, se gestionan con herramientas específicas que registran cada sesión de principio a fin.



2.4.5. Datos

Todos los pilares anteriores existen para proteger esto: los datos. El objetivo final es garantizar que solo las personas correctas, con los dispositivos adecuados, desde los segmentos de red apropiados y a través de los canales controlados, pueden acceder a la información sensible. Aquí entran la clasificación de datos, el cifrado tanto en reposo como en tránsito, y herramientas de prevención de fuga de datos (DLP).

2.5. Zero Trust Network Access (ZTNA)

Vale la pena detenerse un momento en ZTNA porque es el concepto que más directamente conecta el modelo teórico con el problema práctico del acceso remoto, que es precisamente lo que este proyecto aborda.

La VPN tradicional funciona como una llave maestra: si tienes la llave correcta, puedes entrar a la red y moverte por ella. ZTNA funciona de forma completamente diferente: no te da acceso a la red, sino sólo a los recursos concretos que necesitas, y nada más. Aunque tengas credenciales válidas, solo ves lo que se te ha autorizado explícitamente.

Existen dos modelos de implementación. En el modelo iniciado por el cliente, hay un agente en el dispositivo del usuario que se autentica ante un broker antes de establecer el túnel al recurso. En el modelo iniciado por el servicio, no hay agente: el recurso se publica a través de un conector que solo hace conexiones salientes, de modo que el recurso nunca queda expuesto directamente a Internet.

Las soluciones empresariales de referencia son Cloudflare Zero Trust, Zscaler Private Access y BeyondCorp de Google. Para entornos de laboratorio como este, una combinación de OpenSSH con hardening estricto y Fail2ban permite implementar los principios esenciales sin necesidad de infraestructura adicional.

2.6. Casos de Uso y Adopción en la Industria

El modelo Zero Trust lleva años siendo una tendencia en el sector, pero los últimos años han acelerado su adopción de una manera que no tiene precedente. Hay varios factores detrás de esto, y es interesante analizarlos brevemente porque ayuda a entender por qué este proyecto es relevante más allá del ámbito académico.



El caso más evidente fue el teletrabajo masivo durante la pandemia de COVID-19. Las organizaciones que dependían de VPNs tradicionales se encontraron de la noche a la mañana con que su infraestructura no estaba dimensionada para miles de conexiones simultáneas desde casa, y que sus modelos de seguridad asumían una separación entre "dentro" y "fuera" que ya no existía. Las que habían avanzado hacia ZTNA gestionaron la transición de forma mucho más ordenada.

En entornos multicloud, que son ya la norma en empresas medianas y grandes, el modelo Zero Trust encaja de forma natural. Cuando los recursos están distribuidos entre AWS, Azure y aplicaciones SaaS, no hay un "dentro" que proteger: las decisiones de acceso tienen que basarse en identidad y contexto, no en ubicación.

Uno de los argumentos más sólidos a favor de Zero Trust viene de analizar los ciberataques más notorios de los últimos años. El ataque a SolarWinds en 2020, que comprometió a cientos de organizaciones incluyendo agencias gubernamentales americanas, no explotó una vulnerabilidad técnica espectacular: aprovechó la confianza implícita que las redes internas otorgan a sus componentes para moverse lateralmente sin ser detectado durante meses. Una arquitectura con microsegmentación y monitorización continua habría limitado drásticamente el alcance del daño.

Por último, en sectores regulados como el financiero, el sanitario o la administración pública, marcos normativos como PCI-DSS, HIPAA o el Esquema Nacional de Seguridad exigen niveles de control de acceso y trazabilidad que encajan directamente con los principios Zero Trust. Implementar este modelo no es solo una buena práctica de seguridad, es también una forma eficiente de cumplir con la normativa.

2.7. Herramientas y Tecnologías para Implementar Zero Trust

Como se ha dicho antes, Zero Trust no se implementa con una sola herramienta. Se construye combinando soluciones que cubren los diferentes pilares del modelo. A continuación se describen las herramientas elegidas para este proyecto y por qué se eligieron frente a las alternativas.

2.7.1. Control de Acceso Remoto: OpenSSH

OpenSSH es el estándar de facto para el acceso remoto a servidores Linux. No hay mucho debate aquí: es el protocolo SSH en su implementación de referencia, ampliamente auditado, con décadas de uso en producción y una configuración que puede hacerse todo lo estricta que se quiera. Lo que lo hace especialmente relevante para Zero



Trust es su capacidad de deshabilitar la autenticación por contraseña, restringir qué usuarios y grupos pueden conectarse, limitar el número de intentos antes de cerrar la conexión, e integrar un segundo factor de autenticación mediante PAM (desde la versión 6.2). Esta última capacidad es la que permite añadir TOTP a la autenticación SSH, implementando directamente el pilar de identidad del modelo Zero Trust.

2.7.2. Prevención de Intrusiones: Fail2ban

Fail2ban es una de esas herramientas que hace exactamente lo que promete: analiza los logs del sistema en busca de patrones de comportamiento anómalo, como múltiples fallos de autenticación seguidos, y reacciona automáticamente bloqueando la IP origen mediante reglas de cortafuegos. Sin intervención manual. Sin necesidad que un administrador esté mirando los logs en ese momento.

Su configuración gira en torno al concepto de *jail*: cada jail define un servicio a monitorizar, un umbral de intentos fallidos y una duración del bloqueo. La jail sshd es la más directamente relevante para este proyecto: detecta los ataques de fuerza bruta contra SSH y bloquea al atacante antes de que pueda completar el ataque.

2.7.3. Centralización de Logs: rsyslog y Wazuh

Registrar lo que pasa es uno de los principios fundamentales de Zero Trust, pero los logs que solo están en el servidor que se está atacando tienen un problema obvio: si el atacante consigue comprometer ese servidor, puede borrarlos. La solución es enviarlos en tiempo real a un sistema independiente.

rsyslog es el demonio de logging estándar en la mayoría de distribuciones Linux. Con una configuración mínima puede reenviar todos los eventos de seguridad a un servidor centralizado, donde quedan fuera del alcance del atacante.

Wazuh añade la capa de inteligencia sobre esos logs. No se limita a almacenarlos: los analiza en tiempo real, aplica reglas de detección, correlaciona eventos de múltiples fuentes y genera alertas clasificadas por severidad. Su integración con el framework MITRE ATT&CK permite categorizar los ataques según las tácticas y técnicas documentadas, lo que facilita enormemente el análisis forense posterior. En este proyecto, Wazuh es el componente que convierte la telemetría bruta en visibilidad de seguridad real.



2.7.4. Control de Cortafuegos: UFW

UFW (Uncomplicated Firewall) es la interfaz simplificada para iptables en sistemas Ubuntu. No es la herramienta más potente del mercado, pero para este laboratorio es exactamente lo que se necesita: permite configurar una política de denegación por defecto y abrir solo los puertos estrictamente necesarios, con una sintaxis clara y sin complicaciones. En producción se podría optar por nftables o un cortafuegos de siguiente generación, pero para implementar el principio de mínimo privilegio a nivel de red en este contexto, UFW es más que suficiente.

2.7.5. Autenticación Multifactor: Google Authenticator (TOTP)

Google Authenticator implementa el algoritmo TOTP definido en el RFC 6238: genera un código de seis dígitos que cambia cada treinta segundos, basado en una clave compartida y la hora actual. Su integración con SSH se realiza a través del módulo libpam-google-authenticator, que se añade a la cadena de autenticación PAM. El resultado es que, para acceder al servidor, no basta con tener la clave SSH correcta: también hay que introducir el código del momento que genera la aplicación en el móvil. Ambos factores son independientes, así que comprometer uno no compromete el otro.

2.8. Comparativa de Soluciones

La tabla siguiente recoge las principales alternativas para cada capa del stack de seguridad y justifica por qué se eligió cada herramienta. No existe una elección perfecta en seguridad, pero sí elecciones justificadas.

Tabla 1. Comparativa de soluciones por capa.



Capa	Solución elegida	Alternativas	Justificación de la elección
Acceso remoto	OpenSSH	Telnet, RDP, VNC	Estándar de facto en Linux, soporte MFA, ampliamente auditado
Prevención intrusiones	Fail2ban	CrowdSec, DenyHosts, SSHGuard	Madurez, flexibilidad de jails, integración nativa con rsyslog
Centralización logs	Wazuh + rsyslog	ELK Stack, Splunk, Graylog	Stack open source integrado: rsyslog centraliza eventos en tiempo real, Wazuh correlaciona, detecta amenazas y genera alertas clasificadas por MITRE ATT&CK
Cortafuegos	UFW / iptables	nftables, firewallld, pf (BSD)	Simplicidad de gestión, soporte nativo Ubuntu, suficiente para el alcance del proyecto
MFA	Google Authenticator (TOTP)	YubiKey, Duo Security, Authy	Gratuito, estándar abierto (RFC 6238), integración directa con PAM

2.9. Marco Normativo y Estándares Relacionados

Este proyecto no se desarrolla en el vacío. Existe un conjunto de estándares y marcos de referencia que orientan las decisiones de diseño, y cualquier implementación sería debería conocerlos aunque no esté obligada a cumplirlos en su totalidad.

2.9.1. NIST SP 800-207 — Zero Trust Architecture

Publicada en agosto de 2020, esta guía del Instituto Nacional de Estándares y Tecnología de Estados Unidos es la referencia técnica más completa sobre Zero Trust que existe.



Define los componentes lógicos de una arquitectura Zero Trust, los principios que deben guiar el diseño y los diferentes modelos de implementación. Este proyecto la toma como marco de referencia principal, y las decisiones de diseño se han tomado teniendo en cuenta sus recomendaciones.

2.9.2. Esquema Nacional de Seguridad (ENS)

En España, el ENS (regulado por el Real Decreto 311/2022) establece los requisitos de seguridad para los sistemas de información de las administraciones públicas. Aunque este laboratorio no es un sistema público, sus principios de control de acceso, trazabilidad y gestión de incidentes son una referencia útil para evaluar si los controles implementados estarían a la altura de lo que exige la normativa española.

2.9.3. ISO/IEC 27001 — Gestión de la Seguridad de la Información

La ISO 27001 define cómo debe estructurarse un Sistema de Gestión de Seguridad de la Información (SGSI). Sus controles del Anexo A incluye aspectos directamente relacionados con lo que se implementa en este proyecto: control de acceso (A.9), criptografía (A.10), seguridad de las comunicaciones (A.13) y gestión de incidentes (A.16). Alinearse con ISO 27001 no es un objetivo directo de este proyecto, pero sirve como referencia para verificar que los controles implementados son los adecuados.

2.9.4. CIS Controls

Los CIS Controls son un conjunto de acciones de ciberseguridad priorizadas por la comunidad de seguridad. Su versión 8 incluye controles muy concretos que este proyecto cubre de forma directa: configuración segura de software (Control 4), gestión de cuentas (Control 5), control de acceso (Control 6) y gestión de registros de auditoría (Control 8). Es una manera práctica de verificar el nivel de madurez de lo implementado.

3. Diseño de la Infraestructura Virtual

3.1. Visión general del entorno



Para llevar a cabo este proyecto se ha montado un entorno de laboratorio completamente virtualizado sobre VMware Workstation. La idea es simple: replicar a pequeña escala lo que sería una red corporativa real, con sus recursos protegidos, su sistema de monitorización y sus posibles atacantes, todo dentro de un entorno controlado y aislado.

El laboratorio lo forman tres máquinas virtuales que se comunican entre sí a través de una red Host-Only de VMware, usando el rango 192.168.10.0/24. Cada máquina tiene un papel diferente: una actúa como el servidor que queremos proteger, otra centraliza todos los registros de seguridad, y la tercera se usa tanto para comprobar que los accesos legítimos funcionan como para simular ataques y verificar que el sistema responde correctamente.

Usar VMware con red Host-Only tiene una ventaja clara para este tipo de laboratorio: las máquinas están completamente aisladas del exterior. No hay tráfico que pueda salir ni entrar desde Internet, lo que permite ejecutar herramientas de ataque como Hydra o nmap sin ningún riesgo fuera del entorno de pruebas.

3.2. Topología de red y diagrama

La arquitectura sigue una topología en estrella donde todas las máquinas se conectan a través del switch virtual de VMware. En la figura siguiente se puede ver el esquema completo: las tres VMs, sus direcciones IP, qué servicios corre cada una y cómo fluye el tráfico entre ellas.

→ [Figura 1. Diagrama de la infraestructura virtual del laboratorio Zero Trust. \(ver Anexo D\)](#)

Desde el punto de vista lógico se pueden distinguir tres zonas. La primera es donde está el servidor target, que representa cualquier recurso corporativo sensible. La segunda es la central de logs, que recibe y analiza en tiempo real todo lo que pasa en el sistema. Y la tercera es el nodo cliente, que en unas pruebas actúa como usuario legítimo y en otras como atacante, lo que permite comprobar que el sistema distingue un caso del otro aunque ambos vengan del mismo segmento de red.

Ese último punto es especialmente relevante desde la perspectiva Zero Trust: el modelo asume que no se puede confiar en nadie por el hecho de estar dentro de la red. Un usuario interno puede ser igual de peligroso que uno externo, y el sistema tiene que estar preparado para detectarlo.



3.3. Roles de cada nodo

3.3.1. Servidor de recursos (Target) — 192.168.10.20

Este es el activo que se quiere proteger. En una empresa real podría ser un servidor de aplicaciones, de ficheros o de cualquier otro servicio crítico. En el laboratorio se ha configurado con los controles de seguridad propios del modelo Zero Trust:

- OpenSSH con hardening: se deshabilita la autenticación por contraseña, sólo se permite acceso por clave pública, se limita el número de intentos y se añade un segundo factor mediante TOTP.
- Fail2ban: monitoriza el log de autenticación y bloquea de forma automática cualquier IP que falle tres veces seguidas. No hace falta intervención manual, el sistema reacciona solo.
- UFW: cortafuegos con política de denegación por defecto. Solo está abierto el puerto 22 para SSH y el necesario para el envío de logs. El resto se rechaza sin notificar el origen.
- rsyslog en modo agente: reenvía en tiempo real todos los eventos de autenticación y del sistema hacia la central, por UDP en el puerto 514.

En la siguiente captura se puede ver la configuración aplicada en el servidor:

→ [Figura 2. OpenSSH config servidor target. \(ver Anexo D\)](#)

3.3.2. Central de logs (Nodo de monitorización) — 192.168.10.10

Esta máquina es el punto central de toda la monitorización. Recibe los eventos de todos los demás nodos, los procesa y los muestra en un panel de control. Tener los logs en un nodo separado del servidor que se protege no es un detalle menor: si un atacante comprometiera el servidor, los registros de lo que hizo estarían a salvo en otra máquina. Es una práctica básica en cualquier arquitectura de seguridad seria.

Los componentes que corre esta VM son:

- rsyslog en modo servidor: escucha en el puerto UDP 514 y almacena los mensajes recibidos organizados por hostname y fecha.
- Wazuh Manager: plataforma SIEM/XDR que analiza los logs en tiempo real, aplica reglas de detección y genera alertas cuando detecta algo anómalo. Tiene un panel



web accesible desde el navegador donde se pueden ver todos los eventos clasificados por severidad.

- UFW: misma política que en el servidor, solo se permite el tráfico estrictamente necesario.

A continuación se muestra el panel de Wazuh con los eventos recibidos del servidor:

→ [Figura 3. Panel Overview de Wazuh con resumen de alertas por severidad \(192.168.10.10\). \(ver Anexo D\)](#)

→ [Figura 4. Panel de Threat Hunting \(con los contadores de 35 eventos, 7 autenticaciones exitosas, etc.](#)

3.3.3. Nodo de auditoría (Cliente / Atacante) — 192.168.10.30

Este nodo cumple una doble función. Por un lado sirve para comprobar que un acceso legítimo funciona correctamente y queda registrado. Por otro lado, es desde donde se lanzan los ataques en las pruebas, lo que permite verificar que el sistema detecta y bloquea el comportamiento anómalo.

Para los escenarios de ataque se usan dos herramientas:

- nmap: permite descubrir qué hosts están activos en la red y qué puertos tienen abiertos, sin conocer de antemano las IPs. Esto hace el escenario mucho más realista, ya que en un ataque real el atacante no suele saber la dirección del objetivo.
- Hydra: lanza ataques de fuerza bruta automatizados contra el servicio SSH, generando múltiples intentos de autenticación fallidos que Fail2ban y Wazuh deben detectar.

→ [Figura 5. Escaneo de red con nmap desde el nodo cliente \(192.168.10.30\) sin conocer la IP del servidor de antemano. \(ver Anexo D\)](#)

3.4. Especificaciones técnicas

En la tabla siguiente se recogen las características de hardware y software de cada VM. Los recursos se han dimensionado según el rol de cada máquina: la central de logs



necesita más RAM porque Wazuh es bastante exigente, mientras que el servidor y el cliente pueden funcionar bien con menos.

Tabla 2. Especificaciones de las máquinas virtuales.

Parámetro	Servidor (Target)	Central de Logs	Cliente / Atacante
Hostname	servidor	central	cliente
Dirección IP	192.168.10.20	192.168.10.10	192.168.10.30
Máscara de subred	/24 (255.255.255.0)	/24 (255.255.255.0)	/24 (255.255.255.0)
Sistema operativo	Ubuntu Server 24.04 LTS	Ubuntu Server 24.04 LTS	Ubuntu Server 24.04 LTS
RAM asignada	2 GB	4 GB	1 GB
vCPUs	2	2	1
Disco	20 GB	40 GB	20 GB
Red VMware	Host-Only	Host-Only	Host-Only
Rol	Recurso protegido	SIEM / monitorización	Auditoría y ataques

3.5. Configuración de red en VMware

Como ya se ha comentado, la red elegida es Host-Only. Esto crea una red privada que solo es visible desde las VMs y el equipo físico, sin salida al exterior. La razón principal es el aislamiento: usando Hydra o nmap en una red normal podría haber problemas, tanto técnicos como legales. En Host-Only eso no es un riesgo.

La asignación de IPs es estática en todas las máquinas, configuradas mediante Netplan. Se ha elegido el rango 192.168.10.0/24 porque no colisiona con las redes domésticas habituales (192.168.0.x o 192.168.1.x), lo que evita conflictos si el equipo anfitrión está conectado a una red doméstica al mismo tiempo.

→ [Figura 6. Configuración de red estática con Netplan en Ubuntu Server. \(ver Anexo D\)](#)

3.6. Inventario de software

La tabla siguiente recoge todas las herramientas usadas en el laboratorio con su versión y función. No hay ningún software de pago ni propietario: todo es de código abierto, lo que hace el entorno completamente reproducible.



Tabla 3. Inventario de software del laboratorio.

Herramienta	Versión	Para qué se usa
Ubuntu Server	24.04 LTS	Sistema operativo base de las tres VMs
VMware Workstation	17.x	Hipervisor para crear y gestionar las máquinas virtuales
OpenSSH Server	8.9p1	Acceso remoto seguro con hardening Zero Trust
Fail2ban	1.0.2	Detección y bloqueo automático de IPs maliciosas
rsyslog	8.2112	Reenvío de logs al servidor central por UDP 514
Wazuh	4.x	Plataforma SIEM: correlación de eventos, alertas y dashboard
UFW / iptables	0.36	Cortafuegos con política de denegación por defecto
Netplan	0.105	Configuración de IPs estáticas en Ubuntu Server
nmap	7.93	Reconocimiento de red en los escenarios de ataque
Hydra	9.4	Simulación de ataque de fuerza bruta contra SSH

3.7. Flujos de tráfico

En el laboratorio se producen tres tipos de flujos diferenciados, cada uno con su lógica de seguridad.

3.7.1. Acceso legítimo

Cuando el usuario autorizado se conecta al servidor desde el cliente, el proceso es el siguiente: primero OpenSSH verifica la clave pública, luego PAM solicita el código TOTP. Solo si los dos factores son correctos se concede el acceso. Cada paso genera un evento en el log de autenticación, que rsyslog reenvía a la central y Wazuh registra como acceso legítimo.

→ [Figura 7. Acceso SSH exitoso desde el cliente al servidor con autenticación por clave y TOTP. \(ver Anexo D\)](#)

3.7.2. Ataque y respuesta automática

En el escenario de ataque, el proceso empieza con un escaneo nmap para descubrir la IP del servidor, sin conocerla previamente. Una vez identificado el objetivo, Hydra lanza el ataque de fuerza bruta. Fail2ban detecta los tres primeros fallos y bloquea la IP atacante automáticamente mediante una regla en iptables. Al mismo tiempo, rsyslog manda todos esos eventos a la central y Wazuh genera una alerta visible en el dashboard.



→ [Figura 8. Ataque de fuerza bruta con Hydra y bloqueo automático por Fail2ban. \(ver Anexo D\)](#)

3.7.3. Telemetría de seguridad

De forma continua, el servidor reenvía su telemetría hacia la central: eventos de `/var/log/auth.log` y `/var/log/syslog` por UDP 514. Wazuh los indexa y actualiza el dashboard en tiempo real. Este flujo es el que garantiza que el administrador siempre tiene una visión centralizada de lo que pasa, sin tener que entrar máquina por máquina.

→ [Figuras 9. Logs recibidos en la central de logs: verificación con grep en /var/log/syslog. \(ver Anexo D\)](#)

3.8. Decisiones de diseño

Algunas de las decisiones tomadas en este diseño merecen una explicación más detallada, ya que no son arbitrarias sino que responden a criterios de seguridad concretos.

- Nodo de logs separado del servidor: si los logs estuvieran solo en el servidor, un atacante que lo comprometiera podría borrar las evidencias. Al tenerlos en una máquina independiente, se garantiza que persisten aunque el servidor caiga.
- UDP para el reenvío de logs: es más ligero que TCP y no requiere establecer una conexión previa, lo que reduce el impacto en el rendimiento del servidor. En un entorno de producción se usaría TCP con TLS, pero en este laboratorio aislado UDP es suficiente.
- Wazuh en lugar de solo rsyslog: rsyslog centraliza los logs, pero no los analiza. Wazuh añade la capa de correlación, detección y alerta que es lo que realmente implementa el principio Zero Trust de monitorización continua y respuesta automática.
- Escenario de ataque con descubrimiento previo: incluir la fase de reconocimiento con nmap antes del ataque de fuerza bruta hace el escenario mucho más realista. En la práctica, el atacante casi nunca sabe de antemano la IP del objetivo.
- Política deny por defecto en UFW: cualquier puerto que no esté explícitamente abierto se bloquea. Esto implementa directamente el principio de mínimo privilegio a nivel de red.

3.9. Limitaciones del diseño



Este diseño funciona bien como laboratorio y demuestra los principios Zero Trust de forma clara, pero tiene limitaciones obvias respecto a un entorno de producción real que conviene mencionar.

La principal es que todas las máquinas están en el mismo segmento de red. En una empresa real habría segmentos separados para cada zona, con un cortafuegos o router entre ellas que inspeccionaría el tráfico. Aquí la separación es lógica, no física.

Otra limitación es el cifrado del canal de logs. UDP sin cifrar está bien para un entorno aislado, pero en producción habría que usar TCP con TLS para garantizar que nadie puede interceptar o manipular los registros en tránsito.

Por último, la verificación del dispositivo es básica: se comprueba la clave SSH pero no el estado del endpoint. Una implementación más madura incluiría la evaluación del dispositivo (si tiene el SO actualizado, si el antivirus está activo, etc.) como condición adicional para el acceso.

Estas limitaciones no invalidan el laboratorio, sino que abren líneas de trabajo futuro que se desarrollarán en las conclusiones del proyecto.

4. Implementación del Laboratorio

Este capítulo recoge el proceso completo de construcción del laboratorio, desde la creación de las máquinas virtuales hasta la puesta en marcha de todos los servicios de seguridad. Se documenta cada paso con los comandos exactos utilizados y las decisiones tomadas durante la implementación, de forma que el entorno sea completamente reproducible.

4.1. Despliegue de la Infraestructura Virtual

El primer paso fue crear las tres máquinas virtuales en VMware Workstation. Las tres se configuraron con Ubuntu Server 22.04 LTS como sistema operativo base, en red Host-Only para garantizar el aislamiento del entorno.

La configuración de hardware de cada VM se ajustó a su función: la central de logs recibió 4 GB de RAM porque Wazuh necesita recursos suficientes para su motor de



análisis, mientras que el servidor y el cliente funcionan correctamente con 2 GB y 1 GB respectivamente.

→ [Figura 10. Configuración de hardware de la VM servidor en VMware Workstation. \(ver Anexo D\)](#)

4.2. Configuración de Red y Segmentación

Una vez creadas las VMs, el siguiente paso fue asignar direcciones IP estáticas a cada una. Ubuntu Server 24.04 usa Netplan para la configuración de red, y el fichero de configuración se encuentra en `/etc/netplan/50-cloud-init.yaml`.

La configuración es sencilla pero importante: sin IPs estáticas, cada vez que se reinicia una VM podría obtener una IP diferente por DHCP, lo que rompería todas las referencias en los ficheros de configuración de rsyslog, Fail2ban y Wazuh. El rango elegido fue 192.168.10.0/24, asignando la .10 a la central, la .20 al servidor y la .30 al cliente.

La configuración aplicada en el servidor fue la siguiente:

```
network:
  version: 2
  ethernets:
    ens33:
      addresses: [192.168.10.20/24]
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Tras editar el fichero se aplica la configuración con:

```
sudo netplan apply
```

→ [Figura 11. Configuración de red estática con Netplan en el servidor \(192.168.10.20\). \(ver Anexo D\)](#)

4.3. Fortalecimiento (Hardening) del Servicio OpenSSH



Con la red configurada, el siguiente paso fue asegurar el servicio SSH del servidor. Por defecto, OpenSSH en Ubuntu viene con una configuración bastante permisiva que no es adecuada para un entorno Zero Trust. Se realizaron los siguientes cambios:

- Se deshabilitó la autenticación por contraseña, obligando a usar exclusivamente claves criptográficas.
- Se prohibió el acceso directo como usuario root.
- Se redujo el número máximo de intentos de autenticación a 3, lo que hace que Fail2ban actúe más rápidamente.
- Se configuró la autenticación por clave pública como método principal.

El problema con Ubuntu 24.04 es que existe un fichero adicional en `/etc/ssh/sshd_config.d/50-cloud-init.conf` que sobrescribe la configuración principal. Este fichero lo crea automáticamente cloud-init y mantiene PasswordAuthentication en yes aunque se cambie en el fichero principal. La solución fue modificar directamente ese fichero secundario.

Para verificar que la configuración real activa es la correcta, se usa el siguiente comando que muestra los parámetros que SSH está usando en tiempo real, independientemente de en qué fichero estén definidos:

```
sudo sshd -T | grep -E  
"passwordauthentication|pubkeyauthentication|permitrootlogin|maxauthtries"
```

→ [Figura 12. Configuración de hardening SSH activa verificada con sshd -T. \(ver Anexo D\)](#)

Una vez verificada la configuración, se generó el par de claves criptográficas en el cliente y se copió la clave pública al servidor para habilitar la autenticación sin contraseña:

```
ssh-keygen -t ed25519  
# Añadir la clave pública al servidor:  
echo "CLAVE_PUBLICA" >> ~/.ssh/authorized_keys  
chmod 600 ~/.ssh/authorized_keys
```

4.4. Configuración de la Monitorización Centralizada con rsyslog

Para que todos los eventos de seguridad del servidor lleguen a la central en tiempo real, se configuró rsyslog en modo cliente-servidor. rsyslog es el demonio de logging estándar



en Ubuntu y su arquitectura modular permite reenviar mensajes a un servidor remoto con muy poca configuración.

4.4.1. Configuración del servidor rsyslog en la central

En la central se habilitó rsyslog para escuchar conexiones entrantes por UDP en el puerto 514. Se editó el fichero `/etc/rsyslog.conf` y se descomentaron las líneas que activan el módulo UDP:

```
module(load="imudp")
input(type="imudp" port="514")
```

Además se añadió una regla para almacenar los logs recibidos organizados por hostname:

```
$template RemoteLogs, "/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log"
*. * ?RemoteLogs
```

Tras guardar los cambios se reinició el servicio:

```
sudo systemctl restart rsyslog
```

4.4.2. Configuración del agente rsyslog en el servidor

En el servidor se configuró rsyslog para reenviar todos los mensajes de autenticación y del sistema hacia la central. Se añadió al final del fichero `/etc/rsyslog.conf` la siguiente línea:

```
*.* @192.168.10.10:514
```

El símbolo `@` indica que se usa UDP. Si se quisiera TCP se usarían dos arrobas (`@@`). Para verificar que el reenvío funciona se puede usar `tcpdump` en la central para capturar el tráfico UDP entrante en el puerto 514, o simplemente revisar el fichero `/var/log/syslog` de la central y buscar mensajes del servidor:

```
grep "servidor" /var/log/syslog | tail -10
```

→ [Figura 13. Logs del servidor recibidos en la central \(192.168.10.10\) vía rsyslog UDP 514. \(ver Anexo D\)](#)



4.5. Implementación de Reglas de Respuesta Automática con Fail2ban

Fail2ban es la herramienta que implementa la respuesta automática ante comportamientos anómalos, uno de los principios fundamentales del modelo Zero Trust. Su funcionamiento es simple pero efectivo: monitoriza los logs de autenticación en busca de patrones de fallo repetidos y bloquea automáticamente las IPs que los generan mediante reglas de iptables.

La instalación es directa:

```
sudo apt install fail2ban -y
```

La configuración se realiza en el fichero `/etc/fail2ban/jail.local`. Se creó este fichero con la configuración específica para SSH:

```
[sshd]
enabled = true
maxretry = 3
bantime = 3600
findtime = 600
```

Los parámetros clave son: `maxretry = 3` (bloquea tras 3 intentos fallidos), `bantime = 3600` (la IP queda bloqueada durante 1 hora) y `findtime = 600` (el contador se resetea si no hay fallos en 10 minutos).

Para verificar el estado de Fail2ban y ver qué IPs están actualmente baneadas:

```
sudo fail2ban-client status sshd
```

→ [Figura 14. Estado de la jail sshd en Fail2ban mostrando configuración activa. \(ver Anexo D\)](#)

4.6. Instalación y Configuración de Wazuh

Wazuh es la pieza más importante de la arquitectura desde el punto de vista de la monitorización. Se trata de una plataforma SIEM (Security Information and Event Management) y XDR (Extended Detection and Response) de código abierto que proporciona correlación de eventos, detección de amenazas y un panel de control web muy completo.



La instalación se realizó en la central de logs (192.168.10.10) mediante el script oficial de instalación de Wazuh, que despliega automáticamente los tres componentes necesarios: el manager, el indexer (basado en OpenSearch) y el dashboard web.

4.6.1. Instalación del stack completo

Dado que Ubuntu 24.04 no está en la lista de sistemas soportados oficialmente por Wazuh 4.9, fue necesario usar el parámetro `-i` para ignorar la comprobación de compatibilidad:

```
curl -sO https://packages.wazuh.com/4.9/wazuh-install.sh  
sudo bash wazuh-install.sh -a -i
```

El proceso de instalación tardó aproximadamente 15 minutos. Al finalizar, el script mostró las credenciales de acceso al panel web:

```
INFO: You can access the web interface https://<IP>:443  
User: admin  
Password: [contraseña generada automáticamente]
```

Un detalle importante durante la instalación fue el espacio en disco. Wazuh con su indexer OpenSearch consume bastante espacio, y fue necesario ampliar el disco de la VM central de 25 GB a 50 GB mediante las herramientas de VMware y el comando `growpart` de Linux.

4.6.2. Acceso al panel web

El panel de Wazuh es accesible desde cualquier navegador mediante HTTPS. Al ser un certificado autofirmado, el navegador muestra un aviso de seguridad que hay que aceptar antes de acceder. Una vez dentro, el panel muestra un resumen de las alertas de las últimas 24 horas clasificadas por severidad.

En la central, se registró el agente mediante la herramienta `manage_agents`:

```
sudo /var/ossec/bin/manage_agents  
# Opción A: Añadir agente  
# Nombre: servidor, IP: 192.168.10.20  
# Opción E: Extraer clave del agente
```



En el servidor, se añadió el repositorio de Wazuh, se instaló el agente en la versión 4.9.2 (la misma que el manager) y se importó la clave generada en el paso anterior:

```
# Añadir repositorio Wazuh
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg
--no-default-keyring \
  --keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg --import
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] \
  https://packages.wazuh.com/4.x/apt/ stable main" | sudo tee
/etc/apt/sources.list.d/wazuh.list
sudo apt install wazuh-agent=4.9.2-1 -y
```

Después de configurar la IP del manager en el fichero `/var/ossec/etc/ossec.conf` e importar la clave, se arrancó el agente:

```
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

La verificación de que el agente está conectado se realiza tanto desde el servidor (comprobando el log del agente) como desde el panel web de Wazuh:

```
sudo grep -i "connected" /var/ossec/logs/ossec.log | tail -5
```

→ [Figura 15. Panel de Endpoints de Wazuh mostrando el agente “servidor” con estado Active. \(ver Anexo D\)](#)

→ [Figura 16. Panel Threat Hunting de Wazuh con eventos clasificados y gráfica de evolución temporal. \(ver Anexo D\)](#)

4.7. Configuración del Cortafuegos con UFW

El último elemento de la implementación fue configurar UFW en cada máquina con una política de denegación por defecto. Esto implementa directamente el principio Zero Trust de mínimo privilegio a nivel de red: todo el tráfico está bloqueado salvo el que se permite explícitamente.

En el servidor, se configuró UFW para permitir únicamente el tráfico SSH entrante y el reenvío de logs hacia la central:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
```



```
sudo ufw enable
```

En la central, además del SSH, se abrieron los puertos necesarios para Wazuh y rsyslog:

```
sudo ufw allow 22/tcp      # SSH
sudo ufw allow 514/udp     # rsyslog
sudo ufw allow 1514/tcp    # Wazuh agente
sudo ufw allow 1515/tcp    # Wazuh registro
sudo ufw allow 443/tcp     # Panel web Wazuh
sudo ufw enable
```

El efecto de esta configuración es visible en los logs de la central cuando se realiza el escaneo nmap desde el cliente: todos los puertos distintos al 22 aparecen como filtered en nmap y generan entradas UFW BLOCK en los logs, que quedan registradas en la central en tiempo real.

→ [Figura 17. Reglas UFW activas en el servidor mostrando política deny por defecto. \(ver Anexo D\)](#)

4.8. Verificación del Sistema Completo

Una vez configurados todos los componentes, se realizó una verificación completa del sistema para confirmar que todos los servicios estaban activos y comunicándose correctamente.

Desde el servidor se comprobó el estado de todos los servicios:

```
sudo systemctl status ssh
sudo systemctl status fail2ban
sudo systemctl status rsyslog
sudo systemctl status wazuh-agent
```

Todos los servicios aparecieron como active (running), lo que confirma que la implementación es correcta y el sistema está listo para las pruebas.

→ [Figura 18. Estado de todos los servicios de seguridad activos en el servidor. \(ver Anexo D\)](#)



4.9. Implementación de la Autenticación Multifactor (MFA) con TOTP

El último elemento implementado en la pila de seguridad fue la autenticación multifactor mediante TOTP (Time-based One-Time Password), que completa el pilar de identidad del modelo Zero Trust. Hasta este punto, el acceso al servidor requería únicamente la clave privada SSH, lo que constituye un único factor de autenticación. Con la integración de Google Authenticator, el sistema pasa a exigir dos factores independientes: algo que se tiene (la clave privada SSH) y algo que se posee físicamente (el dispositivo móvil con la aplicación generadora de códigos).

4.9.1. Instalación y configuración en el servidor

El primer paso fue instalar el módulo PAM de Google Authenticator en el servidor:

```
sudo apt install libpam-google-authenticator -y
```

A continuación se ejecutó el asistente de configuración con el comando `google-authenticator`, que generó un código QR en la terminal. Este código QR se escaneó con la aplicación Google Authenticator en el dispositivo móvil, estableciendo el secreto compartido TOTP entre el servidor y el móvil. El asistente se configuró con las siguientes opciones: tokens basados en tiempo (TOTP), uso único de cada token para prevenir ataques de repetición, ventana de tiempo estándar de 3 códigos, y rate-limiting habilitado para limitar intentos.

4.9.2. Configuración de PAM y sshd_config

Para integrar el TOTP con SSH fue necesario modificar dos ficheros. En primer lugar, en `/etc/pam.d/sshd` se comentó la línea `@include common-auth` para evitar que PAM solicite contraseña del sistema, y se añadió al final la directiva que activa el módulo TOTP:

```
#@include common-auth  
auth required pam_google_authenticator.so
```

En segundo lugar, en `/etc/ssh/sshd_config` se ajustaron los parámetros para que SSH acepte el método `keyboard-interactive` que permite a PAM solicitar el código TOTP, y se estableció que la autenticación requiere ambos factores en secuencia:

```
KbdInteractiveAuthentication yes  
AuthenticationMethods publickey,keyboard-interactive  
UsePAM yes
```



Tras aplicar estos cambios y reiniciar el servicio SSH con `sudo systemctl restart ssh`, se verificó la configuración activa con el comando `sshd -T`, que confirmó que `passwordauthentication` estaba en `no`, `pubkeyauthentication` en `yes`, `kbdinteractiveauthentication` en `yes`, y `authenticationmethods` en `publickey,keyboard-interactive`.

4.9.3. Verificación del funcionamiento

Desde el cliente se probó la conexión SSH al servidor. El sistema verificó automáticamente la clave pública y a continuación PAM mostró el prompt “Verification code:”, solicitando el código TOTP. Al introducir el código correcto generado por la aplicación en el móvil, el acceso fue concedido. Al introducir un código incorrecto, el sistema rechazó la conexión y tras tres intentos fallidos Fail2ban bloqueó automáticamente la IP del cliente, demostrando la integración completa entre MFA y la respuesta automática ante anomalías.

→ [Figura 19. Autenticación MFA: el sistema solicita el código TOTP y bloquea automáticamente tras códigos incorrectos. \(ver Anexo D\)](#)

5. Pruebas y Resultados

Este capítulo documenta las pruebas realizadas sobre el sistema implementado para validar que cada componente funciona correctamente y que el conjunto cumple con los principios del modelo Zero Trust. Se han definido seis escenarios de prueba que cubren los casos de uso más relevantes: el acceso legítimo con MFA, el reconocimiento de red, el ataque de fuerza bruta, la centralización de logs, la verificación de integridad de logs y la detección en el panel SIEM.

Tabla 4. Resumen de escenarios de prueba.

Escenario	Descripción	Resultado esperado	Resultado obtenido
E1	Acceso SSH legítimo con clave pública + código TOTP (MFA)	Acceso concedido, evento registrado	✓ Correcto
E2	Reconocimiento de red con nmap sin conocer IPs	Hosts descubiertos, puertos identificados	✓ Correcto
E3	Ataque de fuerza bruta SSH con Hydra	Fail2ban bloquea IP tras 3 fallos	✓ Correcto



E4	Análisis de la respuesta del sistema ante el ataque	Eventos de auth.log registrados y centralizados en tiempo real	✓ Correcto
E5	Verificación de logs centralizados en la central	Todos los eventos visibles en central	✓ Correcto
E6	Detección de eventos en panel Wazuh	Alertas clasificadas por severidad	✓ Correcto

5.1. Escenario 1: Acceso Legítimo y Trazabilidad

5.1.1. Descripción del escenario

El primer escenario verifica que un usuario autorizado puede acceder al servidor correctamente con autenticación de dos factores y que ese acceso queda registrado en la central de logs. Desde el nodo cliente (192.168.10.30) se establece una conexión SSH al servidor (192.168.10.20) usando la clave pública previamente configurada más el código TOTP generado por Google Authenticator.

Este escenario es importante porque demuestra que el sistema no solo bloquea accesos no autorizados sino que también registra los legítimos, garantizando la trazabilidad completa de todas las acciones. En Zero Trust, un acceso que no queda registrado es tan problemático como uno que no se valida.

5.1.2. Ejecución

Desde el cliente se ejecuta el siguiente comando:

```
ssh servidor@192.168.10.20
```

El servidor verifica automáticamente la clave pública del cliente y a continuación PAM solicita el código de verificación TOTP generado por Google Authenticator en el dispositivo móvil. Solo si ambos factores son correctos se concede el acceso, lo que demuestra que la autenticación multifactor está correctamente implementada y que PasswordAuthentication está deshabilitado.

→ [Figura 20. Acceso SSH exitoso con autenticación de dos factores: clave pública + código TOTP. \(ver Anexo D\)](#)

5.1.3. Verificación en la central

Simultáneamente, en la central de logs se puede verificar que el evento de acceso llegó correctamente mediante rsyslog:



```
grep "Accepted" /var/log/syslog | tail -5
```

El log muestra el timestamp exacto, el usuario que se autenticó, la dirección IP de origen y el método de autenticación usado (publickey). Esta información es suficiente para reconstruir completamente quién accedió, desde dónde y cuándo.

5.2. Escenario 2: Reconocimiento de Red con nmap

5.2.1. Descripción del escenario

Este escenario simula la fase de reconocimiento que precede a cualquier ataque real. Un atacante que accede a la red interna sin conocimiento previo necesita descubrir qué hosts están activos y qué servicios exponen antes de poder planificar el ataque. Se usa nmap desde el cliente para descubrir la infraestructura sin conocer de antemano las IPs.

Incluir esta fase en las pruebas añade realismo al escenario y demuestra que el sistema detecta no solo el ataque en sí sino también la actividad previa de reconocimiento, que queda registrada en los logs como intentos de conexión bloqueados por UFW.

5.2.2. Ejecución

El primer comando descubre todos los hosts activos en la red sin conocer sus IPs:

```
sudo nmap -sn -PE 192.168.10.0/24
```

El resultado muestra los tres nodos del laboratorio con sus IPs y direcciones MAC. El segundo comando analiza en detalle el servidor descubierto para identificar los servicios que expone:

```
nmap -sV -Pn 192.168.10.20
```

→ [Figura 21. Escaneo de red con nmap descubriendo hosts activos en 192.168.10.0/24. \(ver Anexo D\)](#)

→ [Figura 22. Análisis de puertos y servicios del servidor con nmap -sV. \(ver Anexo D\)](#)

5.2.3. Análisis de resultados

El resultado del escaneo confirma que UFW está haciendo su trabajo correctamente: de los 1000 puertos analizados, 999 aparecen como filtered (bloqueados por el cortafuegos)



y solo el puerto 22 está abierto. Esto es exactamente lo que se espera de una configuración de mínimo privilegio a nivel de red.

Adicionalmente, nmap identifica el servicio como OpenSSH 9.6p1 sobre Ubuntu Linux, lo que demuestra que la enumeración de servicios es posible incluso con UFW activo cuando el puerto está abierto. Esto justifica la importancia de tener Fail2ban configurado: aunque el puerto 22 sea visible, cualquier intento de acceso no autorizado será detectado y bloqueado automáticamente.

5.3. Escenario 3: Simulación de Ataque de Fuerza Bruta

5.3.1. Descripción del escenario

Este es el escenario central del proyecto porque demuestra la capacidad de respuesta automática del sistema ante un ataque real. Se usa Hydra para lanzar un ataque de fuerza bruta contra el servicio SSH del servidor, generando múltiples intentos de autenticación fallidos en rápida sucesión.

A diferencia de las pruebas anteriores que realizamos con la IP del servidor conocida de antemano, en este escenario el atacante parte del resultado del reconocimiento con nmap para identificar el objetivo, lo que hace el escenario completo y realista.

5.3.2. Ejecución del ataque

Desde el cliente se lanza el ataque con un diccionario de contraseñas:

```
hydra -l servidor -P ~/passwords.txt 192.168.10.20 ssh -t 4 -V
```

El parámetro -V muestra cada intento en tiempo real, lo que permite ver exactamente qué contraseñas se están probando y el momento en que el servidor deja de responder como consecuencia del bloqueo por Fail2ban.

→ [Figura 23. Ataque de fuerza bruta con Hydra mostrando los intentos de autenticación fallidos. \(ver Anexo D\)](#)

5.3.3. Respuesta automática de Fail2ban

Mientras Hydra lanza el ataque, en el servidor se puede observar en tiempo real cómo Fail2ban detecta los fallos y actúa. El log de autenticación muestra los intentos fallidos:

```
sudo tail -f /var/log/auth.log
```



Tras el tercer intento fallido, Fail2ban inserta automáticamente una regla en iptables que bloquea toda comunicación procedente de la IP atacante. El estado de la jail confirma el bloqueo:

```
sudo fail2ban-client status sshd
```

→ [Figura 24. Estado de Fail2ban mostrando la IP 192.168.10.30 baneada tras el ataque. \(ver Anexo D\)](#)

5.3.4. Verificación del bloqueo

Para confirmar que el bloqueo es efectivo, se intenta de nuevo la conexión desde el cliente:

```
ssh servidor@192.168.10.20
```

El resultado es "Connection refused", lo que confirma que la IP del cliente está efectivamente bloqueada y no puede establecer ninguna conexión con el servidor, ni siquiera intentar autenticarse.

5.4. Escenario 4: Análisis de la Respuesta del Sistema

5.4.1. Registro de eventos en auth.log

El log de autenticación del servidor registra cada evento con precisión: el timestamp en UTC, el hostname, el proceso que genera el evento, y los detalles del intento de acceso incluyendo la IP de origen y el puerto. Esta granularidad permite reconstruir la secuencia completa del ataque.

→ [Figura 25. Registro de auth.log mostrando los intentos fallidos del ataque de fuerza bruta. \(ver Anexo D\)](#)

5.4.2. Centralización en la central de logs

Todos los eventos generados en el servidor llegan a la central en tiempo real a través de rsyslog. Desde la central se pueden ver tanto los bloqueos de UFW generados por el escaneo nmap como los fallos de autenticación del ataque Hydra, todo en un único fichero de logs:

```
grep "UFW BLOCK" /var/log/syslog | tail -8  
grep "Failed password" /var/log/syslog | tail -8
```



→ [Figura 26. Logs centralizados mostrando eventos UFW BLOCK y Failed password del servidor. \(ver Anexo D\)](#)

Este es uno de los resultados más importantes del proyecto. El hecho de que todos los eventos estén centralizados en un nodo independiente del servidor atacado garantiza que las evidencias persisten aunque el servidor sea comprometido. Un atacante que logre acceder al servidor no puede borrar los registros de su actividad porque estos ya están almacenados en otro sistema al que no tiene acceso.

5.5. Escenario 5: Verificación de la Integridad de los Logs Centralizados

5.5.1. Descripción

Este escenario verifica que la centralización de logs funciona correctamente y que no se pierden eventos durante la transmisión. Se comparan los eventos registrados localmente en el servidor con los recibidos en la central para confirmar que coinciden.

5.5.2. Verificación

En el servidor se cuenta el número de eventos de autenticación fallida registrados localmente:

```
grep "Failed password" /var/log/auth.log | wc -l
```

En la central se hace la misma consulta sobre los logs recibidos del servidor:

```
grep "Failed password" /var/log/syslog | grep "servidor" | wc -l
```

El número de eventos debe coincidir o ser muy similar, confirmando que el reenvío por rsyslog no está perdiendo mensajes. Pequeñas diferencias pueden deberse al tiempo de transmisión entre el momento en que se genera el evento y el momento en que llega a la central, pero en ningún caso deberían ser diferencias significativas.

→ [Figura 27. Comparación del número de eventos en servidor y central para verificar integridad. \(ver Anexo D\)](#)

5.6. Escenario 6: Detección en el Panel Wazuh



5.6.1. Alertas generadas

Wazuh analiza en tiempo real los eventos recibidos del agente instalado en el servidor y genera alertas clasificadas por nivel de severidad. Tras los escenarios de prueba anteriores, el panel muestra un conjunto de alertas que reflejan la actividad del laboratorio. El análisis de los eventos individuales permite identificar las reglas concretas disparadas durante las pruebas. La regla 5716 (sshd: authentication failed, nivel de severidad 5) se activó cada vez que se introdujo un código TOTP incorrecto, registrando la IP de origen 192.168.10.30, el puerto de conexión y el timestamp exacto en UTC. La regla 5710 (sshd: Attempt to login using a non-existent user, nivel 5) se disparó durante los intentos con usuarios no existentes. La regla 5762 (sshd: connection reset, nivel 4) quedó registrada cuando Fail2ban cerró la conexión tras el bloqueo. La regla 5715 (sshd: authentication success, nivel 3) confirmó los accesos legítimos con MFA completado correctamente, y la regla 5501 (PAM: Login session opened, nivel 3) registró la apertura de cada sesión autorizada. Esta granularidad en la correlación de eventos permite reconstruir con precisión la secuencia completa de cualquier incidente.

→ [Figura 28. Lista de eventos en Wazuh mostrando reglas disparadas y nivel de severidad. \(ver Anexo D\)](#)

5.6.2. Threat Hunting

La sección Threat Hunting de Wazuh proporciona una vista más detallada de los eventos, con una gráfica de evolución temporal y una clasificación de los ataques por técnica MITRE ATT&CK. Esta clasificación es especialmente valiosa porque permite identificar patrones de ataque y relacionarlos con técnicas conocidas documentadas en la base de conocimiento de MITRE. En el panel generado durante las pruebas del laboratorio, con 590 eventos totales, 4 fallos de autenticación y 10 autenticaciones exitosas, las técnicas MITRE detectadas fueron las siguientes. La técnica T1110 (Brute Force) y su subtécnica T1110.001 (Password Guessing) quedaron registradas como consecuencia de los intentos repetidos de autenticación fallida desde la IP 192.168.10.30, clasificadas bajo la táctica Credential Access. La técnica T1021 (Remote Services) se activó tanto en los accesos legítimos como en los intentos de ataque, al tratarse de conexiones SSH. La técnica T1078 (Valid Accounts) quedó registrada en los accesos exitosos con clave pública y código TOTP correcto, confirmando que el sistema distingue correctamente entre accesos legítimos y comportamiento anómalo. Esta correlación automática con el framework MITRE ATT&CK elimina la necesidad de clasificar manualmente los eventos y facilita enormemente el análisis forense posterior a cualquier incidente.



→ [Figura 29. Panel Threat Hunting con clasificación MITRE ATT&CK: Brute Force, Password Guessing, SSH y Valid Accounts. \(ver Anexo D\)](#)

5.6.3. Agente activo

El panel de Endpoints confirma que el agente instalado en el servidor está activo y enviando telemetría en tiempo real a la central. Se muestra el nombre del agente, su IP, el sistema operativo, la versión del agente y su estado actual.

→ [Figura 30. Panel Endpoints de Wazuh con el agente servidor en estado Active. \(ver Anexo D\)](#)

5.7. Análisis Global de Resultados

Los seis escenarios de prueba han validado que el sistema implementado cumple con los principios del modelo Zero Trust que se establecieron como objetivos del proyecto. A continuación se analiza cómo cada resultado obtenido se relaciona con dichos principios.

5.7.1. Verificación continua de identidad

El Escenario 1 demuestra que todo acceso al servidor requiere autenticación multifactor de dos factores: clave pública SSH más código TOTP, independientemente de si el usuario se conecta desde dentro de la red o desde fuera. No existe ningún mecanismo de confianza implícita basado en la ubicación de red. Incluso disponiendo de la clave privada SSH, el acceso es imposible sin el código temporal generado en el dispositivo móvil del usuario en ese momento, lo que cumple completamente el pilar de identidad del modelo Zero Trust.

5.7.2. Mínimo privilegio

El Escenario 2 demuestra que UFW bloquea todos los puertos salvo el estrictamente necesario (puerto 22). El servidor no expone ningún servicio adicional que no sea imprescindible para su función, lo que reduce drásticamente la superficie de ataque.

5.7.3. Respuesta automática ante anomalías

El Escenario 3 es la demostración más clara del modelo Zero Trust en acción. El sistema no requiere intervención humana para detectar y responder al ataque: Fail2ban actúa automáticamente en cuanto detecta el patrón de comportamiento anómalo (tres fallos de autenticación consecutivos), bloqueando la IP atacante antes de que pueda completar el ataque.



5.7.4. Trazabilidad completa

Los Escenarios 4 y 5 demuestran que todos los eventos, tanto los legítimos como los maliciosos, quedan registrados en la central de forma centralizada e independiente del servidor protegido. Esto garantiza que las evidencias son íntegras y que cualquier incidente puede ser investigado a posteriori con información completa y fiable.

Tabla 5. Relación entre resultados y principios Zero Trust.

Escenario	Principio Zero Trust validado	Evidencia
E1	Verificación continua de identidad	Acceso concedido con clave pública + código TOTP. Sin ambos factores, acceso denegado
E2	Mínimo privilegio a nivel de red	999 puertos filtrados, solo puerto 22 abierto
E3	Respuesta automática ante anomalías	IP bloqueada automáticamente tras 3 fallos sin intervención manual
E4	Trazabilidad e integridad de registros	Logs centralizados en nodo independiente, resistentes a manipulación
E5	Monitorización continua y correlación	Alertas clasificadas por severidad y técnica MITRE ATT&CK en Wazuh

5.8. Plan de Respuesta ante Incidentes de Seguridad

Una arquitectura de seguridad no está completa si no incluye un protocolo de actuación para cuando algo sale mal. Por muy bien configurado que esté el sistema, ninguna implementación es infalible, y la capacidad de responder de forma ordenada ante un incidente es tan importante como los propios controles preventivos. A continuación se describen dos escenarios de incidente representativos del entorno implementado, con su proceso de gestión completo.

5.8.1. Incidente 1: Compromiso de Credenciales SSH

Descripción del incidente

Un atacante obtiene la clave privada SSH de un usuario legítimo, ya sea mediante robo del dispositivo, phishing o exfiltración de datos, y logra autenticarse en el servidor sorteando los controles habituales. A diferencia de un ataque de fuerza bruta, este acceso no genera fallos de autenticación consecutivos, por lo que Fail2ban no lo detecta automáticamente. Es precisamente este tipo de amenaza el que demuestra la importancia del pilar de identidad del modelo Zero Trust: no basta con controlar el acceso, hay que verificar continuamente quién entra y desde donde.



Proceso de gestión del incidente

Detección: Wazuh genera una alerta de nivel medio al detectar una conexión SSH desde una dirección IP no habitual o fuera del horario de actividad normal del usuario. Los eventos de autenticación quedan registrados en la central de logs con el timestamp exacto, la IP de origen y el usuario afectado. La existencia de una central de logs independiente es clave aquí: si el atacante comprometiera el servidor e intentara borrar evidencias, los registros en la central seguirán intactos.

Registro y clasificación: El incidente se clasifica como crítico dada la naturaleza del acceso comprometido. Se registran todos los detalles disponibles: hora de la primera conexión, comandos ejecutados según los logs de sesión y dirección IP de origen.

Diagnóstico inicial: Se revisan los logs de autenticación en la central filtrando por el hostname del servidor para reconstruir la secuencia completa de acciones del atacante. Se comprueba si hay evidencias de escalada de privilegios, modificación de ficheros críticos o instalación de software no autorizado.

Contención: Se revoca inmediatamente la clave SSH comprometida eliminándola del fichero `~/.ssh/authorized_keys` del servidor. Se bloquea manualmente la IP de origen mediante el comando `sudo fail2ban-client set sshd banip <IP>`. Se deshabilita temporalmente la cuenta del usuario afectado.

Erradicación: Se genera un nuevo par de claves criptográficas para el usuario afectado. Si el atacante hubiera instalado algún artefacto malicioso, se procede a la restauración del servidor desde una imagen limpia. Se verifica la integridad de todos los ficheros de configuración críticos.

Recuperación: Se restaura el acceso al usuario afectado con las nuevas credenciales. Se activan reglas de alerta adicionales en Wazuh para monitorizar con mayor detalle las conexiones de ese usuario durante las semanas siguientes.

Prevención reforzada: Este incidente pone de manifiesto la importancia del segundo factor de autenticación TOTP, ya implementado en este proyecto mediante Google Authenticator. Con MFA activo, el robo de la clave privada no es suficiente para acceder al sistema sin el código temporal del dispositivo móvil del usuario, que expira cada 30 segundos. Esta es la lección más clara que deja este escenario: la combinación de clave pública más TOTP implementada proporciona una defensa en profundidad real frente al compromiso de credenciales.

5.8.2. Incidente 2: Ataque de Fuerza Bruta Distribuido

Descripción del incidente

Un actor malicioso lanza un ataque de fuerza bruta distribuido contra el servicio SSH del servidor desde múltiples direcciones IP simultáneas. A diferencia del escenario de prueba documentado en la sección 5.3, donde el ataque procedía de una única IP que Fail2ban bloqueaba rápidamente tras tres intentos, un ataque distribuido puede intentar superar los



controles utilizando decenas de direcciones diferentes para que ninguna de ellas acumule el número de intentos necesario para activar el bloqueo automático.

Proceso de gestión del incidente

Detección: Wazuh detecta un incremento anómalo en el número de intentos de autenticación fallida procedentes de múltiples IPs en un periodo corto de tiempo. El dashboard muestra un pico de alertas clasificadas con la táctica Credential Access del framework MITRE ATT&CK. Aunque individualmente ninguna IP ha superado el umbral de Fail2ban, el patrón agregado es claramente malicioso y visible desde la central de logs.

Registro y clasificación: El incidente se clasifica como de alto impacto por el riesgo de acceso no autorizado. Se registra el rango de IPs involucradas, los usuarios objetivo del ataque y el periodo de tiempo en que se produjo la actividad anómala.

Diagnóstico inicial: Se analiza el log de autenticación en la central para determinar si algún intento ha tenido éxito. Se comprueba si el ataque sigue activo y a que velocidad se están generando los intentos. Se evalúa si las IPs atacantes pertenecen a rangos conocidos de proxies o redes de bots.

Contención: Se reduce temporalmente el umbral de Fail2ban de 3 a 2 intentos fallidos para aumentar la sensibilidad de detección. Si el ataque es masivo, se considera bloquear el acceso SSH a todo el tráfico exterior salvo desde IPs específicas conocidas, añadiendo una regla de UFW que restrinja el puerto 22 a las IPs autorizadas.

Eradicación: Una vez contenido el ataque, se revisan todas las cuentas de usuario del servidor para verificar que ninguna ha sido comprometida. Se restaura la configuración normal de Fail2ban y se documentan los rangos de IPs atacantes para incluirlos en listas negras permanentes.

Recuperación: Se verifica que el servicio SSH sigue operativo para los usuarios legítimos y que los logs siguen fluyendo correctamente hacia la central. Se revisa el estado de todas las jails de Fail2ban para confirmar que están activas y con la configuración correcta.

Prevención futura: Este incidente refuerza la necesidad de implementar MFA y de configurar umbrales más estrictos en entornos expuestos a internet. También justifica la migración del servicio SSH a un puerto no estándar como medida adicional de ofuscación, y la implementación de geoblocking para restringir el acceso desde regiones geográficas sin usuarios legítimos conocidos.

6. Análisis de Costes y Sostenibilidad

6.1. Coste del Entorno de Laboratorio



Unión Europea

SEDE PORVENIR - VIAPOL: C/ Isabela, 1 - 41013 Sevilla SEDE CARTUJA: C/ Leonardo Da Vinci, 7 - 41092 Sevilla

El laboratorio implementado en este proyecto tiene un coste económico directo de cero euros, ya que se ha construido íntegramente con software de código abierto sobre infraestructura virtualizada en un equipo personal. No se ha utilizado ningún servicio de pago ni licencia propietaria en ninguna de las capas del stack. Esto demuestra que es perfectamente posible montar un entorno de seguridad funcional y representativa sin necesidad de inversión económica, lo cual es especialmente relevante para organizaciones con presupuesto limitado.

La tabla siguiente recoge el coste real de cada componente del laboratorio:

Tabla 6. Coste del entorno de laboratorio.

Componente	Herramienta	Licencia	Coste
Hipervisor	VMware Workstation Pro	Gratuito (uso personal, desde 2024)	0 EUR
Sistema operativo	Ubuntu Server 22.04 LTS	Open source (GPL)	0 EUR
Acceso remoto	OpenSSH	Open source (BSD)	0 EUR
Prevención de intrusiones	Fail2ban	Open source (GPL)	0 EUR
Reenvío de logs	rsyslog	Open source (GPL)	0 EUR
SIEM / XDR	Wazuh	Open source (GPL)	0 EUR
Cortafuegos	UFW / iptables	Open source (GPL)	0 EUR
Autenticación MFA	Google Authenticator	Open source (Apache 2.0)	0 EUR
Total			0 EUR

El único coste indirecto asociado al laboratorio es el consumo eléctrico del equipo anfitrión durante el tiempo de ejecución de las máquinas virtuales, que se estima en menos de 5 kWh para el desarrollo completo del proyecto, lo que supone aproximadamente 1 euro al precio medio de la electricidad en España en 2025.

6.2. Coste de una Implementación Real en Producción

Para contextualizar el valor del proyecto más allá del entorno de laboratorio, resulta útil estimar cuánto costaría implementar una arquitectura equivalente en un entorno corporativo real. Se presentan dos escenarios: una implementación con infraestructura propia (on-premise) y una implementación en la nube pública.

Escenario 1: Infraestructura on-premise



Unión Europea

SEDE PORVENIR - VIAPOL: C/ Isabela, 1 - 41013 Sevilla SEDE CARTUJA: C/ Leonardo Da Vinci, 7 - 41092 Sevilla

En un entorno corporativo con servidores propios, el coste de software seguiría siendo cero al tratarse de herramientas de código abierto. Los costes principales recaerían en el hardware necesario y en el tiempo del personal técnico para la implementación y el mantenimiento continuo del sistema.

Tabla 7. Estimación de costes on-premise para una organización pequeña.

Concepto	Estimación
Servidor para recursos protegidos (amortización anual)	400 - 800 EUR/año
Servidor para central de logs y SIEM (amortización anual)	600 - 1.200 EUR/año
Consumo eléctrico estimado (2 servidores 24/7)	300 - 500 EUR/año
Implementación inicial (técnico junior, aprox. 40 horas)	1.200 - 2.000 EUR
Mantenimiento anual estimado	120 - 200 EUR/año
Total primer año	2.620 - 4.700 EUR
Total años sucesivos	420 - 700 EUR/año

Escenario 2: Infraestructura en nube pública (AWS)

Una implementación equivalente en AWS, usando instancias EC2 para los dos nodos del sistema y almacenamiento EBS, resultaría más económica que la infraestructura propia para organizaciones pequeñas, al eliminar la inversión inicial en hardware y trasladar el coste a un modelo de pago por uso.

Tabla 8. Estimación de costes en AWS para una organización pequeña.

Servicio AWS	Configuración	Coste mensual estimado
EC2 (servidor de recursos)	t3.small, Linux, on-demand	15 - 20 EUR
EC2 (central de logs / Wazuh)	t3.medium, Linux, on-demand	30 - 40 EUR
EBS (almacenamiento)	2 x 20 GB gp3	3 - 5 EUR
Transferencia de datos interna	Tráfico estimado entre nodos	1 - 3 EUR
Total mensual estimado		49 - 68 EUR/mes
Total anual estimado		590 - 820 EUR/año

Para organizaciones más grandes con mayor número de sistemas protegidos, la balanza se inclinaría progresivamente hacia la infraestructura propia, donde el coste marginal de añadir nuevos servidores es menor que en la nube.



6.3. Comparativa con Soluciones Comerciales Equivalentes

A modo de referencia, la tabla siguiente recoge el coste aproximado de soluciones comerciales que cubren funcionalidades equivalentes a las implementadas en este proyecto. La comparativa permite valorar el ahorro que supone optar por herramientas de código abierto sin renunciar a una cobertura funcional comparable.

Tabla 9. Comparativa con soluciones comerciales equivalentes.

Solución comercial	Funcionalidad equivalente	Coste orientativo
CrowdStrike Falcón	SIEM + EDR + respuesta automática	8 - 15 EUR/endpoint/mes
Splunk Enterprise	Centralización y análisis de logs	Desde 1.800 EUR/año
Palo Alto Prisma Access	ZTNA + control de acceso remoto	Desde 3.000 EUR/año
Duo Security (Cisco)	MFA para SSH y aplicaciones	Desde 3 EUR/usuario/mes
IBM QRadar SIEM	SIEM empresarial con correlación	Desde 10.000 EUR/año

La comparativa debe interpretarse con matices. Las soluciones comerciales citadas ofrecen capacidades que van más allá de lo implementado en este proyecto: soporte técnico garantizado con SLAs, inteligencia de amenazas actualizada en tiempo real, equipos de investigación dedicados, escalabilidad gestionada y certificaciones de cumplimiento normativo. CrowdStrike Falcón, por ejemplo, incluye detección basada en inteligencia artificial entrenada con millones de endpoints, algo que Wazuh con su configuración básica no alcanza. Lo que sí demuestra este proyecto es que los principios fundamentales del modelo Zero Trust —autenticación multifactor, mínimo privilegio, monitorización centralizada y respuesta automática— pueden implementarse con herramientas de código abierto a coste prácticamente nulo. Esto lo convierte en una opción especialmente válida para organizaciones con presupuesto limitado, entornos académicos o como punto de partida antes de escalar a soluciones empresariales más completas.

6.4. Sostenibilidad

Desde el punto de vista de la sostenibilidad, la arquitectura propuesta presenta varias características positivas que merece la pena destacar.

En primer lugar, al tratarse de un entorno virtualizado, no requiere hardware dedicado durante el desarrollo y las pruebas, lo que reduce el impacto ambiental asociado a la fabricación y el consumo energético de equipos físicos. Tres máquinas virtuales corriendo



sobre un único equipo anfitrión consumen significativamente menos energía que tres servidores físicos independientes.

En segundo lugar, el uso exclusivo de software de código abierto elimina la dependencia de proveedores y garantiza la continuidad del sistema independientemente de decisiones comerciales externas. Herramientas como Wazuh, rsyslog o Fail2ban llevan décadas en desarrollo activo respaldadas por comunidades amplias, lo que ofrece garantías sólidas de mantenimiento a largo plazo.

Por último, la arquitectura es modular y puede desplegarse sobre infraestructura existente sin necesidad de adquirir nuevo hardware, alargando la vida útil de los equipos en uso y reduciendo la generación de residuos electrónicos. En un contexto en que la sostenibilidad tecnológica es cada vez más relevante para las organizaciones, estos factores representan una ventaja adicional frente a las soluciones propietarias que requieren hardware certificado o actualizaciones periódicas forzadas. Cabe señalar, no obstante, que los entornos SIEM en producción tienen un impacto energético no despreciable: plataformas como Wazuh con su indexer OpenSearch pueden consumir recursos considerables en términos de CPU y memoria cuando se escalan a cientos de agentes. En este proyecto, al tratarse de un entorno de laboratorio virtualizado con un único agente, el consumo se estima en menos de 5 kWh para el desarrollo completo, lo que equivale aproximadamente a 1 euro al precio medio de la electricidad en España en 2025. En un despliegue real de mayor escala, optimizar la retención de logs y el dimensionamiento del indexer sería una decisión relevante tanto desde el punto de vista económico como medioambiental.

7. Conclusiones y Líneas Futuras

7.1. Conclusiones

Este proyecto nació con un objetivo concreto: demostrar que es posible implementar un sistema de control de acceso basado en el modelo Zero Trust usando exclusivamente herramientas de código abierto, sobre infraestructura virtual, y sin necesidad de hardware especializado ni licencias de software propietario. Los resultados obtenidos confirman que ese objetivo se ha alcanzado.

A lo largo del desarrollo del proyecto se ha construido un entorno de laboratorio completo y funcional formado por tres máquinas virtuales que replican los roles de una red corporativa real: un servidor de recursos que actúa como activo protegido, una central de monitorización que centraliza y analiza toda la telemetría de seguridad, y un nodo de auditoría desde el que se han ejecutado tanto los accesos legítimos como los ataques simulados.



Sobre esta infraestructura se ha implementado una pila de seguridad que cubre los principales pilares del modelo Zero Trust: la identidad mediante autenticación multifactor de dos factores (clave pública SSH más código TOTP via Google Authenticator) con la autenticación por contraseña completamente deshabilitada, la red mediante cortafuegos UFW con política de denegación por defecto, la respuesta automática ante anomalías mediante Fail2ban, y la trazabilidad y monitorización mediante rsyslog centralizado y la plataforma SIEM Wazuh.

7.1.1. Objetivos alcanzados

Revisando los objetivos específicos planteados al inicio del proyecto, todos han sido cubiertos satisfactoriamente:

- Se ha implementado un sistema de autenticación multifactor (MFA) que combina clave pública SSH con código TOTP mediante Google Authenticator, eliminando la posibilidad de acceso con un único factor comprometido. El servidor no acepta autenticación por contraseña y exige los dos factores en secuencia en cada conexión.
- Se ha configurado un mecanismo de respuesta automática con Fail2ban que bloquea las IPs atacantes sin necesidad de intervención manual, en un tiempo inferior al necesario para completar un ataque de diccionario estándar.
- Se ha centralizado toda la telemetría de seguridad en un nodo independiente del servidor protegido mediante rsyslog, garantizando que las evidencias de cualquier incidente persisten incluso en caso de compromiso del servidor.
- Se ha desplegado la plataforma Wazuh como solución SIEM/XDR que proporciona correlación de eventos, clasificación de alertas por severidad, y visualización de ataques mediante el framework MITRE ATT&CK.
- Se ha validado el funcionamiento del sistema mediante cinco escenarios de prueba que cubren tanto los casos de uso legítimos como los ataques simulados, obteniendo los resultados esperados en todos ellos.

7.1.2. Lecciones aprendidas

El desarrollo del proyecto ha permitido adquirir conocimiento práctico sobre aspectos que en la literatura académica suelen abordarse de forma abstracta. Configurar y depurar la interacción entre rsyslog, Fail2ban y Wazuh en un entorno real ha puesto de manifiesto que la integración de herramientas de seguridad es considerablemente más compleja que



su instalación individual, y que la mayor parte de los problemas no están en la configuración de cada herramienta sino en la comunicación entre ellas.

También ha quedado demostrado que el modelo Zero Trust no es un producto que se compra e instala sino una filosofía de diseño que requiere tomar decisiones conscientes en cada capa de la arquitectura. Deshabilitar la autenticación por contraseña, configurar UFW con política de denegación por defecto, o separar el almacenamiento de logs del servidor que los genera son decisiones que individualmente parecen menores pero que en conjunto producen un sistema cualitativamente más resistente que uno configurado con los valores por defecto.

7.1.3. Limitaciones del proyecto

El sistema implementado es un laboratorio de demostración, no un sistema de producción, y presenta limitaciones que conviene reconocer explícitamente. La principal es la escala: el entorno funciona con tres máquinas virtuales en la misma red, mientras que una organización real tendría decenas o cientos de sistemas distribuidos en múltiples segmentos de red.

Otras limitaciones relevantes son la ausencia de cifrado en el canal de reenvío de logs (se usa UDP sin TLS), la verificación de dispositivo limitada a la validación de clave SSH y código TOTP sin evaluación automatizada de la postura de seguridad del endpoint (estado del SO, antivirus, indicadores de compromiso), y la falta de un sistema de identidad centralizado como LDAP o Active Directory que permitiría una gestión de usuarios más escalable.

7.2. Logros Adicionales y Líneas Futuras

Esta sección recoge, en primer lugar, los logros técnicos que se plantearon inicialmente como líneas futuras pero que finalmente se implementaron durante el desarrollo del proyecto. A continuación se describen las líneas de trabajo que quedan abiertas para evolucionar el sistema hacia una implementación más próxima a un entorno de producción real, ordenadas de mayor a menor viabilidad a corto plazo.

7.2.1. Cifrado del canal de telemetría

La mejora más inmediata sería configurar el reenvío de logs de rsyslog usando TCP con TLS en lugar de UDP sin cifrar. Esto garantiza la confidencialidad e integridad de los



registros durante la transmisión, evitando que un atacante con acceso a la red pueda interceptar o manipular los eventos antes de que lleguen a la central. rsyslog soporta TLS de forma nativa mediante el módulo gtls.

7.2.2. Autenticación Multifactor (MFA) — Implementada

La autenticación multifactor basada en TOTP mediante Google Authenticator y el módulo libpam-google-authenticator ha sido implementada en este proyecto como parte del pilar de identidad del modelo Zero Trust. Con MFA activo, el acceso al servidor requiere dos factores independientes: la clave privada SSH (algo que se tiene) y el código temporal de seis dígitos generado por la aplicación en el móvil (algo que se posee físicamente). Incluso si un atacante consiguiera la clave privada SSH del usuario, no podría acceder al servidor sin el código TOTP del momento, cuya validez expira cada 30 segundos. La implementación quedó validada en el Escenario 1 de pruebas, donde se verifica que el sistema solicita el código de verificación en cada conexión y rechaza códigos incorrectos activando automáticamente el bloqueo de Fail2ban.

7.2.3. Evaluación de postura de dispositivo

Una implementación Zero Trust más madura incluiría la evaluación del estado del dispositivo como condición adicional para el acceso. Esto implicaría verificar que el dispositivo tiene el sistema operativo actualizado, que no presenta indicadores de compromiso conocidos, y que cumple con las políticas de configuración corporativas. Herramientas como el agente Wazuh con sus módulos de System Inventory y SCA (Security Configuration Assessment) permiten aproximarse a esta capacidad.

7.2.4. Microsegmentación real

La segmentación de red implementada en este proyecto es lógica: todas las máquinas están en la misma subred y la separación se consigue mediante reglas de UFW en cada host. Una implementación más robusta separaría físicamente cada zona en una VLAN diferente, con un cortafuegos entre ellas que inspeccionara todo el tráfico de este a oeste. Esto impediría el movimiento lateral incluso en caso de compromiso de uno de los nodos.

7.2.5. Integración con un directorio de identidades

La gestión de usuarios basada en cuentas locales de Linux no escala bien en organizaciones con muchos empleados. La integración con un servidor LDAP o Active Directory permitiría gestionar las identidades de forma centralizada, aplicar políticas de contraseña corporativas y desactivar cuentas de forma inmediata cuando un empleado



abandona la organización, aspectos todos ellos críticos desde la perspectiva del pilar de identidad del modelo Zero Trust.

7.2.6. Alertas automáticas y respuesta orquestada

Wazuh soporta la integración con sistemas de ticketing y plataformas SOAR (Security Orchestration, Automation and Response) que permiten automatizar la respuesta a incidentes más allá del simple bloqueo de IP que proporciona Fail2ban. Por ejemplo, ante la detección de un ataque sofisticado, el sistema podría automáticamente abrir un ticket en Jira, notificar al equipo de seguridad por correo o Slack, y aplicar reglas de bloqueo adicionales en el cortafuegos perimetral.

7.3. Reflexión Final

El modelo Zero Trust no es una moda ni una solución mágica. Es una forma de pensar la seguridad que parte de asumir que cualquier sistema puede ser comprometido y que, por tanto, el diseño debe minimizar el impacto de ese compromiso. Lo que este proyecto ha demostrado es que sus principios fundamentales son aplicables con herramientas accesibles, sin grandes presupuestos y sin necesidad de infraestructura especializada.

La seguridad no se consigue instalando un producto, sino tomando buenas decisiones de diseño de forma consistente: deshabilitar lo que no hace falta, verificar lo que sí hace falta, registrar todo lo que ocurre y reaccionar rápido cuando algo sale mal. Eso es Zero Trust, y eso es lo que este proyecto ha implementado y validado.

8. Bibliografía y Anexos

8.1. Bibliografía

A continuación se recogen las referencias bibliográficas consultadas durante la elaboración de este trabajo, ordenadas por categoría temática.

8.1.1. Estándares y marcos normativos

- [1] Rose, S., Borchert, O., Mitchell, S. y Connelly, S. (2020). Zero Trust Architecture. NIST Special Publication 800-207. National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-207>



- [2] Centro Criptológico Nacional (2022). Esquema Nacional de Seguridad. Real Decreto 311/2022, de 3 de mayo. Boletín Oficial del Estado.
- [3] International Organization for Standardization (2022). ISO/IEC 27001:2022 — Information security, cybersecurity and privacy protection. ISO.
- [4] Center for Internet Security (2021). CIS Controls v8. Center for Internet Security. <https://www.cisecurity.org/controls/v8>

8.1.2. Publicaciones académicas y técnicas

- [5] Saltzer, J. H. y Schroeder, M. D. (1975). The protection of information in computer systems. Proceedings of the IEEE, 63(9), 1278-1308.
- [6] Kindervag, J. (2010). No More Chewy Centers: Introducing The Zero Trust Model Of Information Security. Forrester Research.
- [7] Ward, R. y Beyer, B. (2014). BeyondCorp: A New Approach to Enterprise Security. USENIX ;login:, 39(6), 6-11.
- [8] Gilman, E. y Barth, D. (2017). Zero Trust Networks: Building Secure Systems in Untrusted Networks. O'Reilly Media.

8.1.3. Documentación técnica de herramientas

- [9] Wazuh Inc. (2024). Wazuh Documentation — Installation guide. <https://documentation.wazuh.com/current/installation-guide/index.html>
- [10] OpenSSH Project (2024). OpenSSH manual pages — sshd_config(5). https://man.openbsd.org/sshd_config
- [11] Fail2ban Project (2024). Fail2ban documentation. https://www.fail2ban.org/wiki/index.php/Main_Page
- [12] rsyslog Project (2024). rsyslog documentation. <https://www.rsyslog.com/doc/index.html>
- [13] Canonical Ltd. (2024). Ubuntu Server Guide — Networking. <https://ubuntu.com/server/docs/network-configuration>
- [14] MITRE Corporation (2024). MITRE ATT&CK Framework. <https://attack.mitre.org/>

8.1.4. Recursos web complementarios

- [15] National Security Agency (2021). Embracing a Zero Trust Security Model. NSA Cybersecurity Information Sheet.



https://media.defense.gov/2021/Feb/25/2002588479/-1/-1/0/CSI_EMBRACING_ZT_SECURITY_MODEL_UOO115131-21.PDF

[16] CISA (2022). Zero Trust Maturity Model. Cybersecurity and Infrastructure Security Agency. <https://www.cisa.gov/zero-trust-maturity-model>

[17] Cloudflare (2024). What is Zero Trust security? <https://www.cloudflare.com/learning/security/glossary/what-is-zero-trust/>

8.2. Anexos

Anexo A. Ficheros de configuración completos

A continuación se recogen los ficheros de configuración más relevantes del laboratorio para facilitar la reproducibilidad del entorno.

A.1. Configuración de OpenSSH (/etc/ssh/sshd_config.d/50-cloud-init.conf)

```
# Hardening Zero Trust – OpenSSH
PasswordAuthentication no
PubkeyAuthentication yes
PermitRootLogin no
MaxAuthTries 3
AuthenticationMethods publickey
```

A.2. Configuración de Fail2ban (/etc/fail2ban/jail.local)

```
[DEFAULT]
bantime  = 3600
findtime = 600
maxretry = 3

[sshd]
enabled = true
port    = ssh
logpath = /var/log/auth.log
backend = systemd
```



A.3. Configuración de rsyslog en el servidor (/etc/rsyslog.d/99-remote.conf)

```
# Reenvío de logs a la central
*. * @192.168.10.10:514
```

A.4. Configuración de rsyslog en la central (/etc/rsyslog.conf — fragmento)

```
# Módulo UDP
module(load="imudp")
input(type="imudp" port="514")

# Almacenamiento por hostname
$template RemoteLogs, "/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log"
*. * ?RemoteLogs
```

A.5. Configuración de Netplan — Servidor (/etc/netplan/50-cloud-init.yaml)

```
network:
  versión: 2
  ethernet:
    ens33:
      addresses: [192.168.10.20/24]
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

A.6. Reglas UFW — Servidor

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw enable
```

A.7. Reglas UFW — Central de logs

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 514/udp
sudo ufw allow 1514/tcp
sudo ufw allow 1515/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```



Anexo B. Comandos de verificación del sistema

Comandos útiles para verificar el estado del sistema y reproducir las pruebas documentadas en el Capítulo 5.

B.1. Verificar configuración activa de SSH

```
sudo sshd -T | grep -E  
"passwordauthentication|pubkeyauthentication|permitrootlogin|maxauthtries"
```

B.2. Ver estado de Fail2ban

```
sudo fail2ban-client status sshd
```

B.3. Desbanear una IP manualmente

```
sudo fail2ban-client set sshd unbanip 192.168.10.30
```

B.4. Verificar que los logs llegan a la central

```
grep "servidor" /var/log/syslog | tail -20
```

B.5. Escaneo de red con nmap

```
sudo nmap -sn -PE 192.168.10.0/24  
nmap -sV -Pn 192.168.10.20
```

B.6. Ataque de fuerza bruta con Hydra

```
hydra -l servidor -P ~/passwords.txt 192.168.10.20 ssh -t 4 -V
```

B.7. Verificar estado del agente Wazuh

```
sudo systemctl status wazuh-agent  
sudo grep -i "connected" /var/ossec/logs/ossec.log | tail -5
```

B.8. Ver todos los servicios de seguridad activos

```
sudo systemctl status ssh fail2ban rsyslog wazuh-agent
```



Anexo C. Glosario de términos

SIEM: Security Information and Event Management. Plataforma que centraliza y correlacionar eventos de seguridad de múltiples fuentes para detectar amenazas.

XDR: Extended Detection and Response. Evolución del SIEM que integra detección y respuesta en múltiples capas de la infraestructura.

Zero Trust: Modelo de seguridad basado en el principio de no confiar en ningún usuario o dispositivo por defecto, verificando cada acceso independientemente de su origen.

ZTNA: Zero Trust Network Access. Modelo de acceso remoto que aplica los principios Zero Trust, concediendo acceso únicamente a los recursos específicos autorizados.

MFA: Multi-Factor Authentication. Autenticación que requiere dos o más factores independientes para verificar la identidad del usuario.

TOTP: Time-based One-Time Password. Algoritmo que genera códigos de un solo uso con validez temporal, definido en el RFC 6238.

SSH: Secure Shell. Protocolo criptográfico para el acceso remoto seguro a sistemas Unix/Linux.

UFW: Uncomplicated Firewall. Interfaz simplificada para la gestión de iptables en sistemas Ubuntu/Debian.

rsyslog: Demonio de logging estándar en Linux con soporte para reenvío remoto de mensajes de log.

Fail2ban: Herramienta de prevención de intrusiones que analiza logs y bloquea IPs con comportamiento anómalo.

Wazuh: Plataforma SIEM/XDR de código abierto para monitorización de seguridad, detección de amenazas y respuesta a incidentes.

MITRE ATT&CK: Base de conocimiento de tácticas y técnicas de ataque documentadas, usada como referencia para clasificar y analizar amenazas.

PAM: Pluggable Authentication Modules. Framework de Linux que permite configurar mecanismos de autenticación de forma modular.



ENS: Esquema Nacional de Seguridad. Marco normativo español que establece los requisitos de seguridad para sistemas de información de las administraciones públicas.

DLP: Data Loss Prevention. Conjunto de herramientas y procesos para detectar y prevenir la exfiltración de datos sensibles.



Unión Europea

SEDE PORVENIR - VIAPOL: C/ Isabela, 1 - 41013 Sevilla SEDE CARTUJA: C/ Leonardo Da Vinci, 7 - 41092 Sevilla

Anexo D. Capturas de pantalla

Este anexo recoge todas las capturas de pantalla referenciadas en el cuerpo del documento. Cada figura incluye su número, descripción y la imagen correspondiente.

Figura 1. Diagrama de la infraestructura virtual del laboratorio Zero Trust.

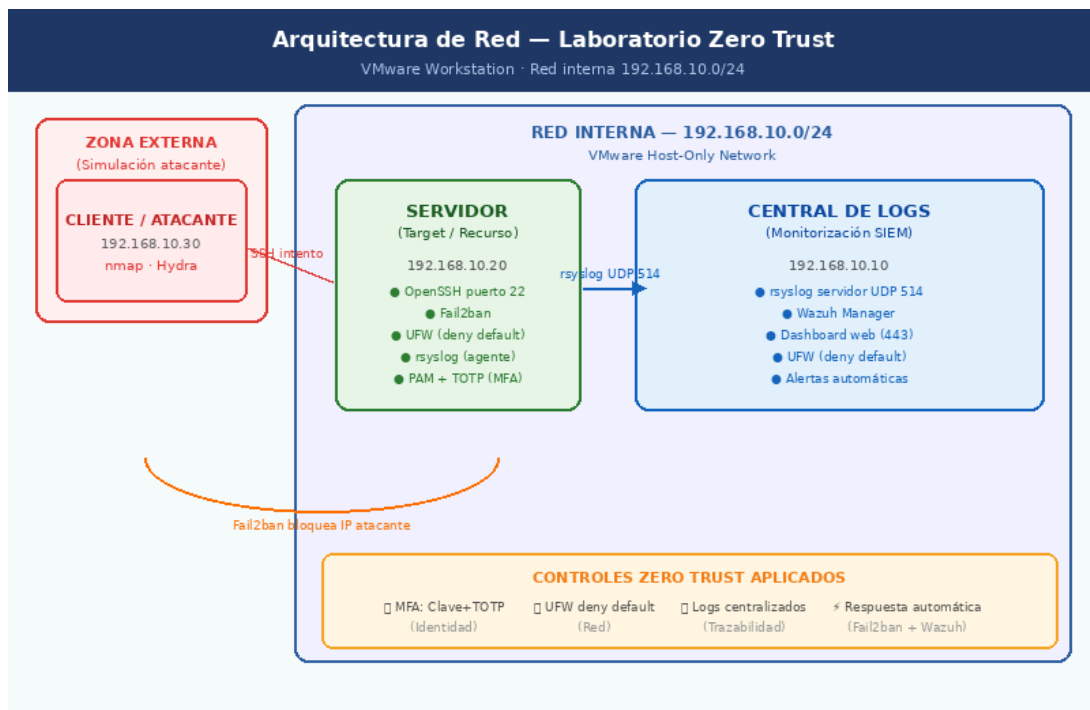


Figura 1. Diagrama de la infraestructura virtual del laboratorio Zero Trust.

Figura 2. OpenSSH config servidor target.

```
servidor@servidor:~$ sudo ss
maxauthtries 3
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
servidor@servidor:~$ |
```



Figura 2. OpenSSH config servidor target

Figura 3. Panel Overview de Wazuh con resumen de alertas por severidad (192.168.10.10).

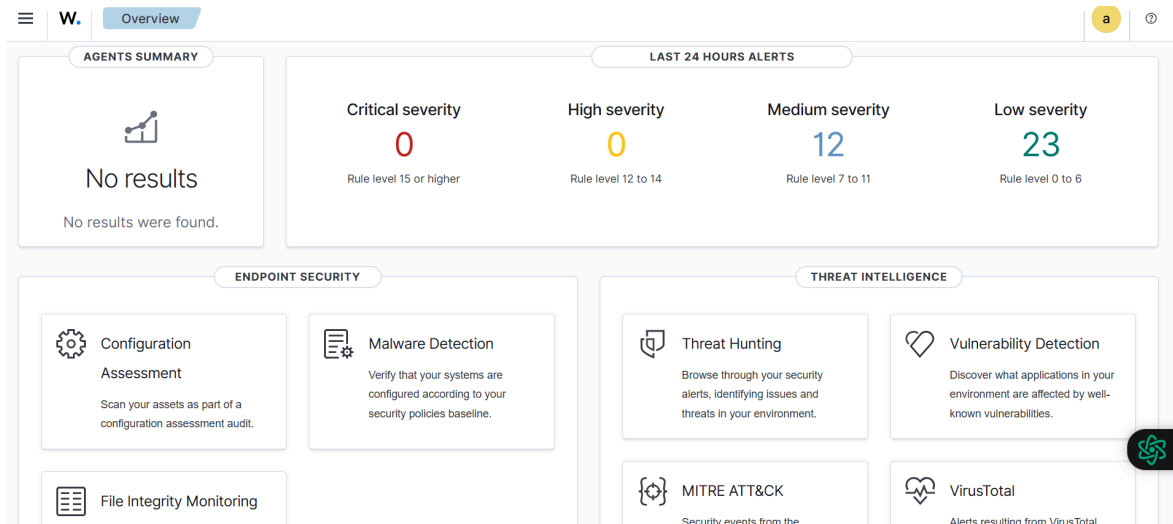


Figura 3. Panel Overview de Wazuh con resumen de alertas por severidad (192.168.10.10).

Figura 4. Panel de Threat Hunting (con los contadores de 35 eventos, 7 autenticaciones exitosas, etc.

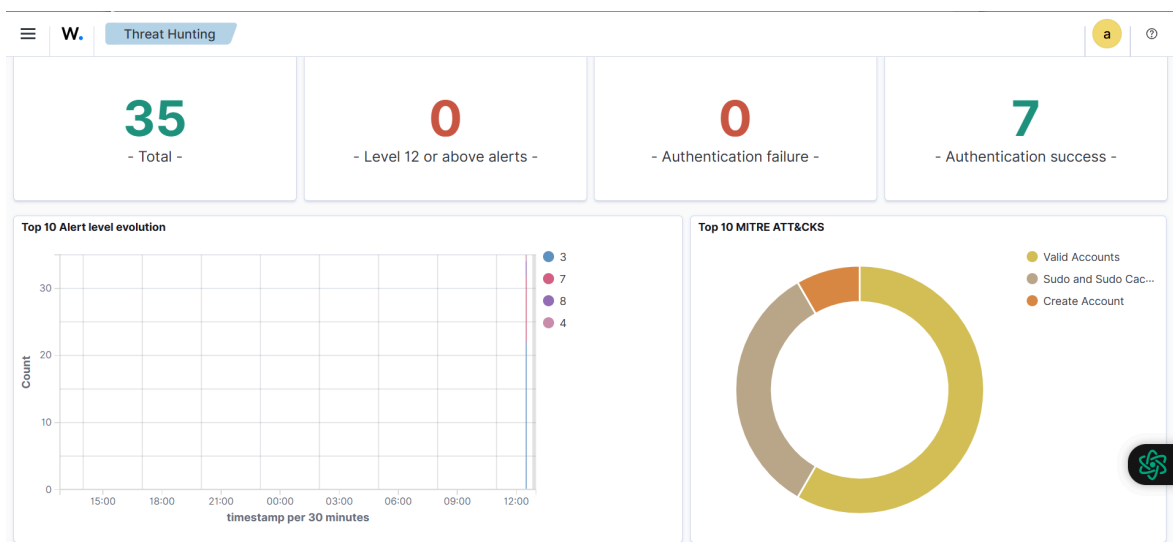


Figura 4. Panel de Threat Hunting (con los contadores de 35 eventos, 7 autenticaciones exitosas, etc.



Unión Europea

SEDE PORVENIR - VIAPOL: C/ Isabela, 1 - 41013 Sevilla SEDE CARTUJA: C/ Leonardo Da Vinci, 7 - 41092 Sevilla

Figura 5. Escaneo de red con nmap desde el nodo cliente (192.168.10.30) sin conocer la IP del servidor de antemano.

```
cliente@cliente:~$ sudo nmap -sn -PE 192.168.10.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-04 10:15 UTC
Nmap scan report for 192.168.10.1
Host is up (0.00027s latency).
MAC Address: 00:50:56:C0:00:01 (VMware)
Nmap scan report for 192.168.10.10
Host is up (0.00022s latency).
MAC Address: 00:0C:29:AA:B2:9A (VMware)
Nmap scan report for 192.168.10.20
Host is up (0.00086s latency).
MAC Address: 00:0C:29:E5:1B:18 (VMware)
Nmap scan report for cliente (192.168.10.30)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.15 seconds
cliente@cliente:~$
```

```
cliente@cliente:~$ nmap -sV -Pn 192.168.10.20
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-04 10:17 UTC
Nmap scan report for 192.168.10.20
Host is up (0.00083s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux; protocol 2.0)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 8.31 seconds
cliente@cliente:~$
```

Figura 5. Escaneo de red con nmap desde el nodo cliente (192.168.10.30) sin conocer la IP del servidor de antemano.

Figura 6. Configuración de red estática con Netplan en Ubuntu Server.

```
servidor@servidor:~$ sudo cat /etc/netplan/50-cloud-init.yaml
network:
  version: 2
  ethernets:
    ens33:
      addresses: [192.168.10.20/24]
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
servidor@servidor:~$
```



Figura 6. Configuración de red estática con Netplan en Ubuntu Server (archivo /etc/netplan/00-installer-config.yaml).

Figura 7. Acceso SSH exitoso desde el cliente al servidor con autenticación por clave y TOTP.

```
cliente@cliente:~$ ssh servidor@192.168.10.20
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-110-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of dom 03 may 2026 18:22:45 UTC

System load: 0.0          Memory usage: 8%    Processes:      220
Usage of /: 33.7% of 19.51GB Swap usage: 0%    Users logged in: 1
```

Figura 7. Acceso SSH exitoso desde el cliente al servidor con autenticación por clave y TOTP.

Figura 8. Ataque de fuerza bruta con Hydra y bloqueo automático por Fail2ban.

```
servidor@servidor:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed: 4
| '- File list: /var/log/auth.log
'- Actions
  |- Currently banned: 1
  |- Total banned: 1
  '- Banned IP list: 192.168.10.30
servidor@servidor:~$
```

```
cliente@cliente:~$ ssh servidor@192.168.10.20
ssh: connect to host 192.168.10.20 port 22: Connection refused
cliente@cliente:~$
```

Figura 8. Ataque de fuerza bruta con Hydra y bloqueo automático por Fail2ban. Se puede ver el estado de la jail sshd.



Figuras 9. Logs recibidos en la central de logs: verificación con grep en /var/log/syslog.

```

central@central:~$ grep "UFW BLOCK" /var/log/syslog | tail -8
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=37045 DF PROTO=TCP SPT=44420 DPT=23 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=42086 DF PROTO=TCP SPT=49352 DPT=113 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=10384 DF PROTO=TCP SPT=43010 DPT=993 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=14941 DF PROTO=TCP SPT=59088 DPT=1723 WINDOW=64240 RE
S=0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=7292 DF PROTO=TCP SPT=38568 DPT=80 WINDOW=64240 RES=0
x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=11420 DF PROTO=TCP SPT=41510 DPT=25 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=37045 DF PROTO=TCP SPT=44420 DPT=23 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=42086 DF PROTO=TCP SPT=49352 DPT=113 WINDOW=64240 RES=
0x00 SYN URGP=0
central@central:~$

```

Eventos de bloqueo UFW registrados en la central desde el servidor (192.168.10.20)

```

central@central:~$ grep "Failed password" /var/log/syslog | tail -5
2026-04-30T15:09:06+00:00 servidor sshd[14030]: Failed password for servidor from 192.168.10.30 port 34104 ssh2
2026-04-30T15:09:17+00:00 servidor sshd[14030]: message repeated 2 times: [ Failed password for servidor from 192.168.10
.30 port 34104 ssh2]
2026-04-30T15:14:57+00:00 servidor sshd[14063]: Failed password for servidor from 192.168.10.30 port 39648 ssh2
2026-04-30T15:15:09+00:00 servidor sshd[14068]: Failed password for servidor from 192.168.10.30 port 57594 ssh2
2026-05-04T10:18:37.046381+00:00 central sshd[1109]: Failed password for invalid user cliente from 192.168.10.1 port 515
87 ssh2
central@central:~$

```

Intentos de autenticación fallida generados por el ataque de Hydra, registrados en la central via syslog

Figuras 9. Logs recibidos en la central de logs: verificación con grep en /var/log/syslog.**Figura 10. Configuración de hardware de la VM servidor en VMware Workstation.**

Unión Europea

SEDE PORVENIR - VIAPOL: C/ Isabela, 1 - 41013 Sevilla SEDE CARTUJA: C/ Leonardo Da Vinci, 7 - 41092 Sevilla

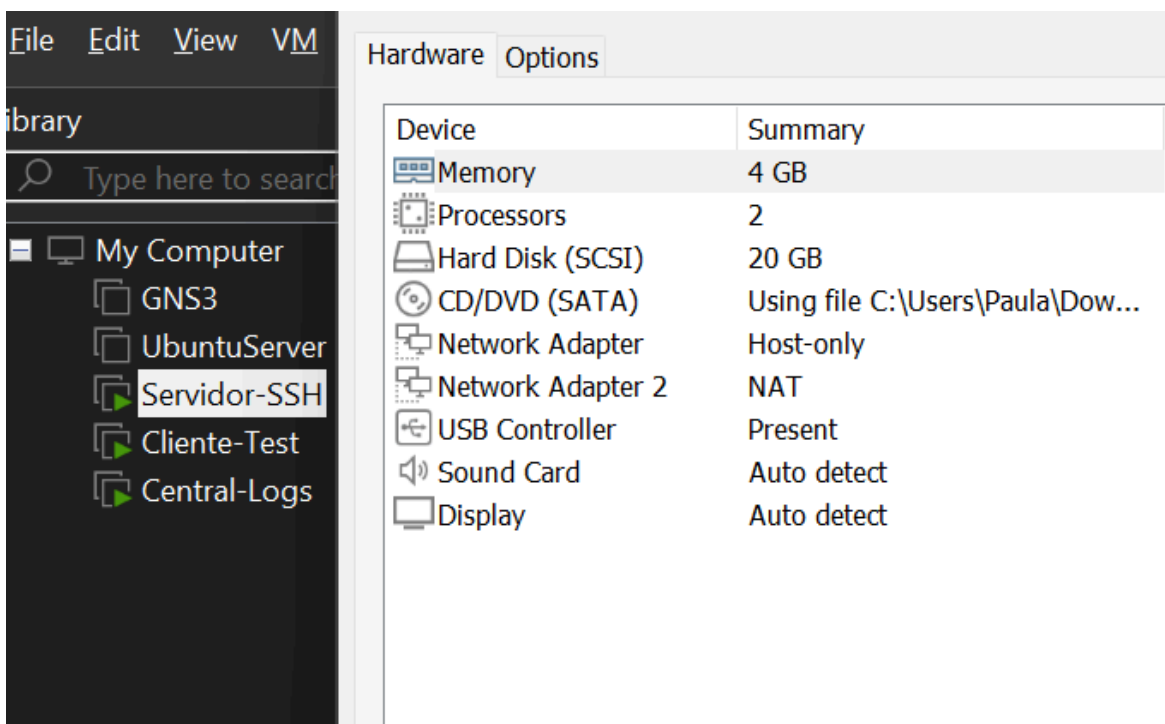


Figura 10. Configuración de hardware de la VM servidor en VMware Workstation.

Figura 11. Configuración de red estática con Netplan en el servidor (192.168.10.20).

```
servidor@servidor:~$ sudo cat /etc/netplan/50-cloud-init.yaml
network:
  version: 2
  ethernets:
    ens33:
      addresses: [192.168.10.20/24]
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
servidor@servidor:~$
```

Figura 11. Configuración de red estática con Netplan en el servidor (192.168.10.20).

Figura 12. Configuración de hardening SSH activa verificada con sshd -T.



```

servidor@servidor:~$ sudo ss
maxauthtries 3
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
servidor@servidor:~$ |

```

Figura 12. Configuración de hardening SSH activa verificada con `sshd -T`.

Figura 13. Logs del servidor recibidos en la central (192.168.10.10) vía rsyslog UDP 514.

```

central@central:~$ grep "UFW BLOCK" /var/log/syslog | tail -8
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=37045 DF PROTO=TCP SPT=44420 DPT=23 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=42086 DF PROTO=TCP SPT=49352 DPT=113 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=10384 DF PROTO=TCP SPT=43010 DPT=993 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=14941 DF PROTO=TCP SPT=59088 DPT=1723 WINDOW=64240 RE
S=0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=7292 DF PROTO=TCP SPT=38568 DPT=80 WINDOW=64240 RES=0
x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=11420 DF PROTO=TCP SPT=41510 DPT=25 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=37045 DF PROTO=TCP SPT=44420 DPT=23 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=42086 DF PROTO=TCP SPT=49352 DPT=113 WINDOW=64240 RES=
0x00 SYN URGP=0
central@central:~$

```

Figura 13. Logs del servidor recibidos en la central (192.168.10.10) vía rsyslog UDP 514.

Figura 14. Estado de la jail sshd en Fail2ban mostrando configuración activa.



Unión Europea

SEDE PORVENIR - VIAPOL: C/ Isabela, 1 - 41013 Sevilla SEDE CARTUJA: C/ Leonardo Da Vinci, 7 - 41092 Sevilla

```
servidor@servidor:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed:      4
| `-- File list:        /var/log/auth.log
`- Actions
  |- Currently banned: 1
  |- Total banned:      1
  `-- Banned IP list:   192.168.10.30
servidor@servidor:~$
```

Figura 14. Estado de la jail sshd en Fail2ban mostrando configuración activa.

Figura 15. Panel de Endpoints de Wazuh mostrando el agente “servidor” con estado Active.

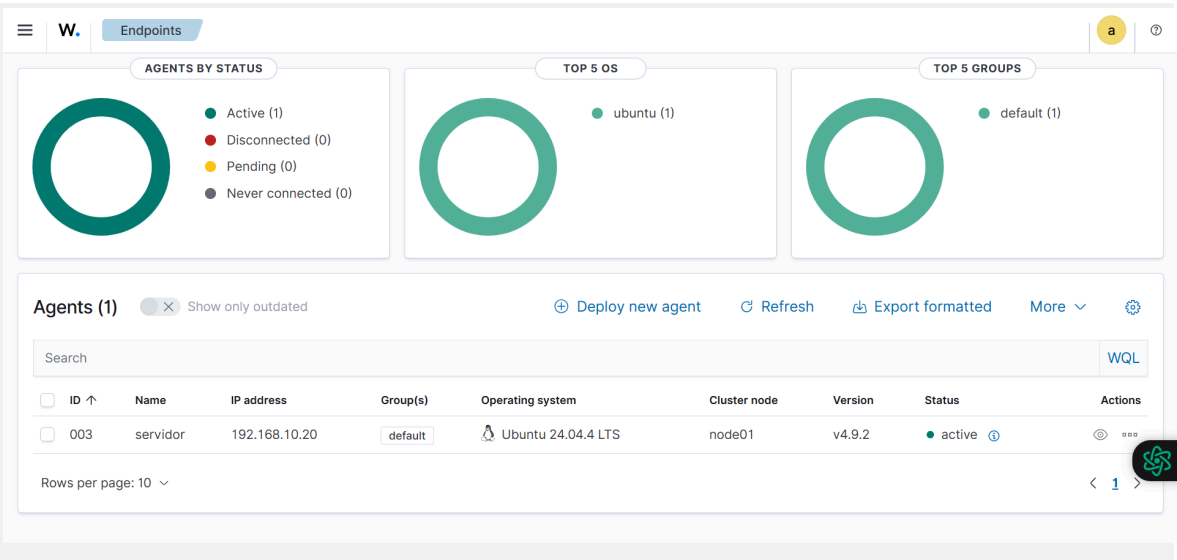


Figura 15. Panel de Endpoints de Wazuh mostrando el agente "servidor" con estado Active.

Figura 16. Panel Threat Hunting de Wazuh con eventos clasificados y gráfica de evolución temporal.



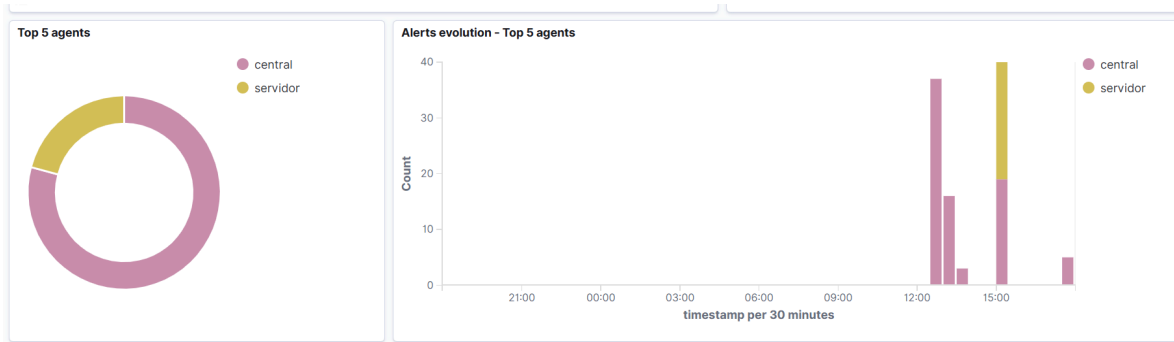


Figura 16. Panel Threat Hunting de Wazuh con eventos clasificados y gráfica de evolución temporal.

Figura 17. Reglas UFW activas en el servidor mostrando política deny por defecto.

```
servidor@servidor:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
514/udp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
514/udp (v6) ALLOW IN Anywhere (v6)

servidor@servidor:~$
```

Figura 17. Reglas UFW activas en el servidor mostrando política deny por defecto.

Figura 18. Estado de todos los servicios de seguridad activos en el servidor.



```

● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-05-04 11:20:07 UTC; 4h 44min ago
   TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
   Main PID: 1570 (sshd)
     Tasks: 1 (limit: 4543)
    Memory: 3.3M (peak: 19.7M)
       CPU: 66ms
   CGroup: /system.slice/ssh.service
           └─1570 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

may 04 11:20:07 servidor systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
may 04 11:20:07 servidor sshd[1570]: Server listening on 0.0.0.0 port 22.
may 04 11:20:07 servidor systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
may 04 11:20:07 servidor sshd[1570]: Server listening on :: port 22.
may 04 11:20:28 servidor sshd[1572]: Accepted password for servidor from 192.168.150.1 port 65246 ssh2
may 04 11:20:28 servidor sshd[1572]: pam_unix(sshd:session): session opened for user servidor(uid=1000) by servidor(uid=0)

● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-05-04 11:11:19 UTC; 4h 53min ago
     Docs: man:fail2ban(1)
   Main PID: 860 (fail2ban-server)
     Tasks: 5 (limit: 4543)
    Memory: 19.0M (peak: 19.5M)
       CPU: 9.188s
   CGroup: /system.slice/fail2ban.service
           └─860 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

```

Figura 18. Estado de todos los servicios de seguridad activos en el servidor.

Figura 19. Autenticación MFA: el sistema solicita el código TOTP y bloquea automáticamente tras códigos incorrectos.

```

cliente@cliente:~$ ssh servidor@192.168.10.20
(servidor@192.168.10.20) Verification code:
(servidor@192.168.10.20) Verification code:
(servidor@192.168.10.20) Verification code:
^C
cliente@cliente:~$ ssh servidor@192.168.10.20
ssh: connect to host 192.168.10.20 port 22: Connection refused
cliente@cliente:~$

```

Figura 19. Autenticación MFA: el sistema solicita el código TOTP tras verificar la clave pública, y bloquea automáticamente la IP tras códigos incorrectos repetidos.

Figura 20. Acceso SSH exitoso con autenticación de dos factores: clave pública + código TOTP.



```
cliente@cliente:~$ ssh servidor@192.168.10.20
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-110-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of dom 03 may 2026 18:22:45 UTC

System load: 0.0           Memory usage: 8%    Processes:      220
Usage of /:  33.7% of 19.51GB Swap usage:   0%    Users logged in: 1
```

Figura 20. Acceso SSH exitoso desde el cliente al servidor con autenticación de dos factores: clave pública + código TOTP (Verification code).

Figura 21. Escaneo de red con nmap descubriendo hosts activos en 192.168.10.0/24.

```
cliente@cliente:~$ sudo nmap -sn -PE 192.168.10.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-04 10:15 UTC
Nmap scan report for 192.168.10.1
Host is up (0.00027s latency).
MAC Address: 00:50:56:C0:00:01 (VMware)
Nmap scan report for 192.168.10.10
Host is up (0.00022s latency).
MAC Address: 00:0C:29:AA:B2:9A (VMware)
Nmap scan report for 192.168.10.20
Host is up (0.00086s latency).
MAC Address: 00:0C:29:E5:1B:18 (VMware)
Nmap scan report for cliente (192.168.10.30)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.15 seconds
```

Figura 21. Escaneo de red con nmap descubriendo los hosts activos en 192.168.10.0/24.

Figura 22. Análisis de puertos y servicios del servidor con nmap -sV.



```

cliente@cliente:~$ nmap -sV -Pn 192.168.10.20
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-04 10:17 UTC
Nmap scan report for 192.168.10.20
Host is up (0.00083s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux; protocol 2.0)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 8.31 seconds
cliente@cliente:~$

```

Figura 22. Análisis de puertos y servicios del servidor con nmap -sV.

Figura 23. Ataque de fuerza bruta con Hydra mostrando los intentos de autenticación fallidos.

```

cliente@cliente:~$ hydra -l servidor -P ~/passwords.txt 192.168.10.20 ssh -t 4 -V
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret
or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-04 09:59:35
[DATA] max 4 tasks per 1 server, overall 4 tasks, 10 login tries (l:1/p:10), ~3 tries per task
[DATA] attacking ssh://192.168.10.20:22/
[ATTEMPT] target 192.168.10.20 - login "servidor" - pass "123456" - 1 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.10.20 - login "servidor" - pass "password" - 2 of 10 [child 1] (0/0)
[ATTEMPT] target 192.168.10.20 - login "servidor" - pass "1234" - 3 of 10 [child 2] (0/0)
[ATTEMPT] target 192.168.10.20 - login "servidor" - pass "servidor" - 4 of 10 [child 3] (0/0)
[ATTEMPT] target 192.168.10.20 - login "servidor" - pass "admin" - 5 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.10.20 - login "servidor" - pass "root" - 6 of 10 [child 3] (0/0)
[ATTEMPT] target 192.168.10.20 - login "servidor" - pass "test" - 7 of 10 [child 1] (0/0)
[ATTEMPT] target 192.168.10.20 - login "servidor" - pass "password123" - 8 of 10 [child 2] (0/0)
[ATTEMPT] target 192.168.10.20 - login "servidor" - pass "abc123" - 9 of 10 [child 3] (0/0)
[ATTEMPT] target 192.168.10.20 - login "servidor" - pass "letmein" - 10 of 10 [child 0] (0/0)
[STATUS] 10.00 tries/min, 10 tries in 00:01h, 1 to do in 00:01h, 2 active
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-04 10:00:43
cliente@cliente:~$

```

Figura 23. Ataque de fuerza bruta con Hydra mostrando los intentos de autenticación fallidos.

Figura 24. Estado de Fail2ban mostrando la IP 192.168.10.30 baneada tras el ataque.



```

servidor@servidor:~$ sudo fail2ban-client status sshd
[sudo] password for servidor:
Status for the jail: sshd
|- Filter
|   |- Currently failed: 1
|   |- Total failed: 6
|   \- File list: /var/log/auth.log
- Actions
    |- Currently banned: 1
    |- Total banned: 2
    \- Banned IP list: 192.168.10.30
servidor@servidor:~$ _

```

Figura 24. Estado de Fail2ban mostrando la IP 192.168.10.30 baneada tras el ataque.

Figura 25. Registro de auth.log mostrando los intentos fallidos del ataque de fuerza bruta.

```

servidor@servidor:~$ sudo grep "Failed\\|authentication failure" /var/log/auth.log | tail -15
2026-05-04T09:55:50.053409+00:00 servidor passwd[1517]: pam_unix(passwd:chauthtok): authentication failure; logname=serv
idor uid=1000 euid=0 tty= ruser= rhost= user=servidor
2026-05-04T09:59:35.316673+00:00 servidor sshd[1556]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0
tty=ssh ruser= rhost=192.168.10.30 user=servidor
2026-05-04T09:59:35.342753+00:00 servidor sshd[1555]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0
tty=ssh ruser= rhost=192.168.10.30 user=servidor
2026-05-04T09:59:35.368465+00:00 servidor sshd[1553]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0
tty=ssh ruser= rhost=192.168.10.30 user=servidor
2026-05-04T09:59:35.368722+00:00 servidor sshd[1554]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0
tty=ssh ruser= rhost=192.168.10.30 user=servidor
2026-05-04T09:59:37.220070+00:00 servidor sshd[1556]: Failed password for servidor from 192.168.10.30 port 34872 ssh2
2026-05-04T09:59:37.246755+00:00 servidor sshd[1555]: Failed password for servidor from 192.168.10.30 port 34858 ssh2
2026-05-04T09:59:37.272781+00:00 servidor sshd[1553]: Failed password for servidor from 192.168.10.30 port 34840 ssh2
2026-05-04T09:59:37.273580+00:00 servidor sshd[1554]: Failed password for servidor from 192.168.10.30 port 34848 ssh2
2026-05-04T11:11:35.983838+00:00 servidor login[994]: pam_unix(login:auth): authentication failure; logname=LOGIN uid=0
euid=0 tty=/dev/tty1 ruser= rhost= user=servidor
2026-05-04T11:13:10.222491+00:00 servidor sudo: pam_unix(sudo:auth): authentication failure; logname=servidor uid=1000 e
uid=0 tty=/dev/tty1 ruser=servidor rhost= user=servidor
2026-05-05T18:09:46.735045+00:00 servidor sshd[14169]: Failed keyboard-interactive/pam for servidor from 192.168.10.30 p
ort 41488 ssh2
2026-05-06T12:11:16.218983+00:00 servidor sshd[2825]: Failed keyboard-interactive/pam for servidor from 192.168.10.30 po
rt 42090 ssh2
2026-05-06T12:43:27.361475+00:00 servidor sshd[2956]: Failed keyboard-interactive/pam for servidor from 192.168.10.30 po
rt 47120 ssh2
2026-05-06T12:45:40.383047+00:00 servidor sudo: servidor : TTY=pts/1 ; PWD=/home/servidor ; USER=root ; COMMAND=/usr/bin
/grep 'Failed\\|authentication failure' /var/log/auth.log
servidor@servidor:~$

```

Figura 25. Registro de auth.log mostrando los intentos fallidos del ataque de fuerza bruta.

Figura 26. Logs centralizados mostrando eventos UFW BLOCK y Failed password del servidor.



```

central@central:~$ grep "UFW BLOCK" /var/log/syslog | tail -8
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=37045 DF PROTO=TCP SPT=44420 DPT=23 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=42086 DF PROTO=TCP SPT=49352 DPT=113 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=10384 DF PROTO=TCP SPT=43010 DPT=993 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=14941 DF PROTO=TCP SPT=59088 DPT=1723 WINDOW=64240 RE
S=0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=7292 DF PROTO=TCP SPT=38568 DPT=80 WINDOW=64240 RES=0
x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=11420 DF PROTO=TCP SPT=41510 DPT=25 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=37045 DF PROTO=TCP SPT=44420 DPT=23 WINDOW=64240 RES=
0x00 SYN URGP=0
2026-05-04T10:17:18+00:00 servidor kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00:0c:29:e5:1b:18:00:0c:29:eb:8b:ee:08:00 SRC=1
92.168.10.30 DST=192.168.10.20 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=42086 DF PROTO=TCP SPT=49352 DPT=113 WINDOW=64240 RES=
0x00 SYN URGP=0
central@central:~$

```

Figura 26. Logs centralizados en la central mostrando eventos UFW BLOCK y Failed password del servidor.

Figura 27. Comparación del número de eventos en servidor y central para verificar integridad.

```

central@central:~$ grep -a "Failed password" /var/log/syslog | grep "servidor" | wc -l
4
central@central:~$

servidor@servidor:~$ grep "Failed password" /var/log/auth.log | wc -l
7
servidor@servidor:~$

```

Figura 27. Comparación del número de eventos en servidor y central para verificar integridad de logs.

Figura 28. Lista de eventos en Wazuh mostrando reglas disparadas y nivel de severidad.



May 5, 2026 @ 14:04:28.299 - May 6, 2026 @ 14:04:28.299					
Export Formatted 475 columns hidden Density 1 fields sorted Full screen					
timestamp	agent.name	rule.description	rule.level	rule.id	
May 6, 2026 @ 13:59:34.016	servidor	sshd: authentication failed.	5	5716	
May 6, 2026 @ 13:59:33.989	servidor	sshd: Attempt to login using a no...	5	5710	
May 6, 2026 @ 13:59:33.973	servidor	sshd: connection reset	4	5762	
May 6, 2026 @ 13:59:33.973	servidor	sshd: connection reset	4	5762	
May 6, 2026 @ 13:59:33.934	servidor	PAM: Login session opened.	3	5501	
May 6, 2026 @ 13:59:33.934	servidor	sshd: authentication success.	3	5715	
May 6, 2026 @ 13:59:33.934	servidor	sshd: authentication success.	3	5715	
May 6, 2026 @ 13:59:33.888	servidor	sshd: Attempt to login using a no...	5	5710	
May 6, 2026 @ 13:59:33.868	servidor	sshd: Attempt to login using a no...	5	5710	
May 6, 2026 @ 13:59:33.835	servidor	sshd: authentication success.	3	5715	
May 6, 2026 @ 13:59:33.835	servidor	PAM: Login session opened.	3	5501	
May 6, 2026 @ 13:59:33.835	servidor	sshd: connection reset	4	5762	
May 6, 2026 @ 13:59:33.835	servidor	sshd: authentication success.	3	5715	

Figura 28. Lista de eventos en Wazuh mostrando reglas disparadas, nivel de severidad e IDs durante las pruebas del laboratorio.

Figura 29. Panel Threat Hunting con clasificación MITRE ATT&CK: Brute Force, Password Guessing, SSH y Valid Accounts.



Figura 29. Panel Threat Hunting de Wazuh mostrando distribución de alertas por nivel de severidad y clasificación MITRE ATT&CK: Brute Force, Password Guessing, SSH y Valid Accounts.

Figura 30. Panel Endpoints de Wazuh con el agente servidor en estado Active.



Unión Europea

SEDE PORVENIR - VIAPOL: C/ Isabela, 1 - 41013 Sevilla SEDE CARTUJA: C/ Leonardo Da Vinci, 7 - 41092 Sevilla

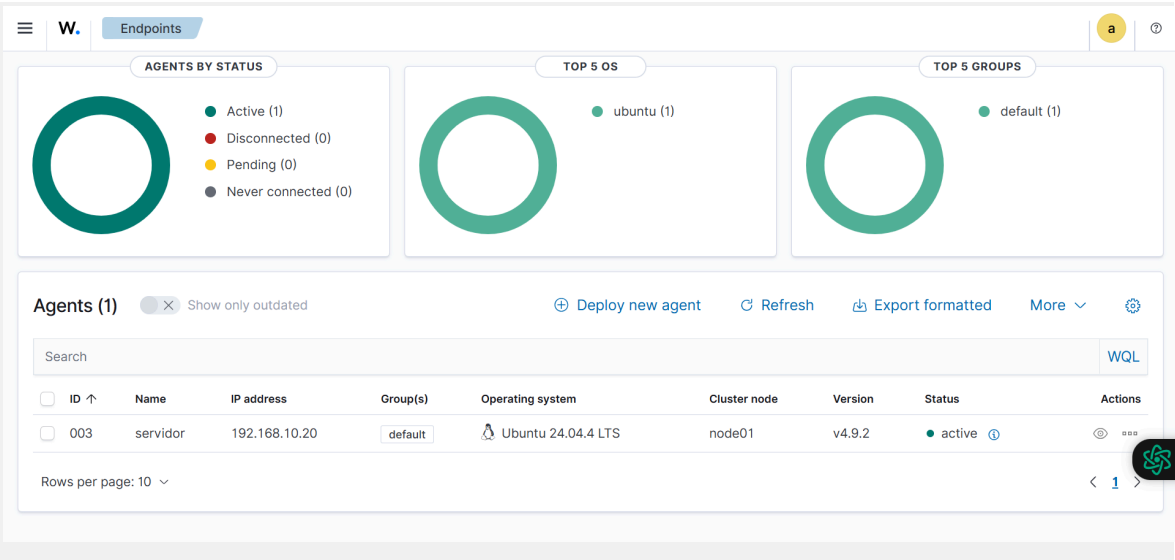


Figura 30. Panel Endpoints de Wazuh con el agente servidor en estado Active.

